

A decorative graphic on the left side of the slide consists of six hexagons arranged in a staggered, overlapping pattern. The hexagons are colored orange, teal, dark blue, light green, dark blue, and red. Each hexagon has a thin white border and a slight drop shadow, giving it a 3D appearance.

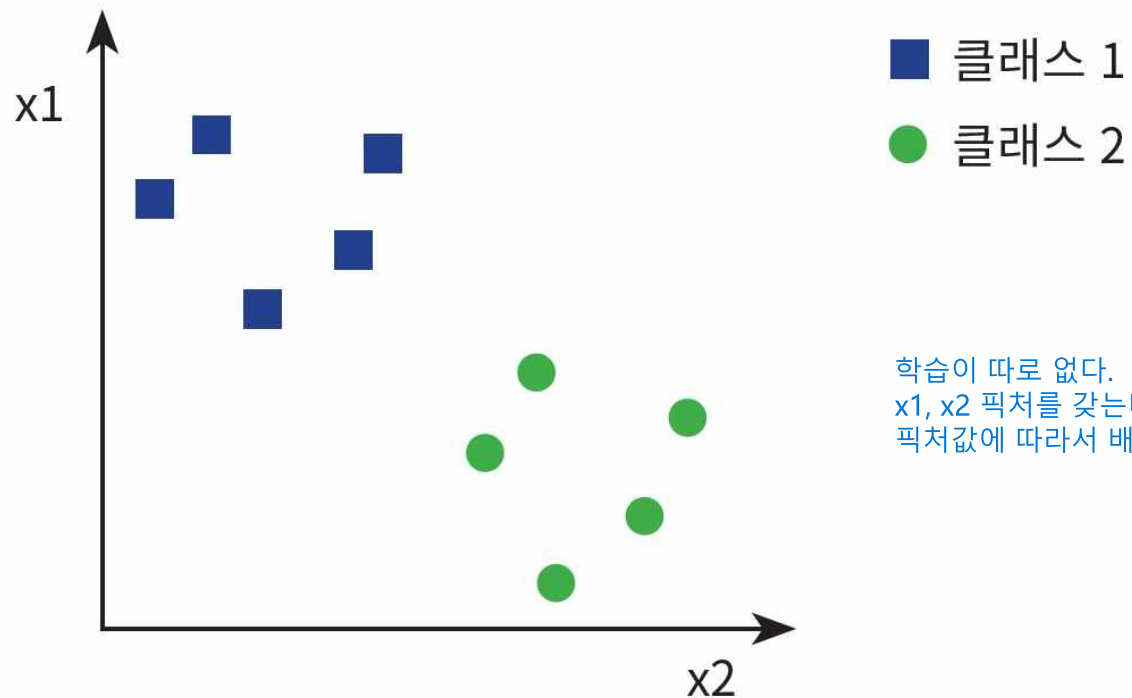
제11장 KNN, K-MEANS

학습 목표

- kNN 알고리즘을 이해한다. *k Nearest Neighbors* k개의 가까운 이웃을 가지는 알고리즘
- K-means 알고리즘을 이해한다. mean은 수학에서 평균을 의미한다 (k개의 평균으로 나눔)
- 결정 트리를 이해한다.
- 비지도 학습을 이해한다.
- sklearn을 사용하여 kNN 알고리즘과 K-means 알고리즘을 구현할 수 있다.

01 kNN 알고리즘

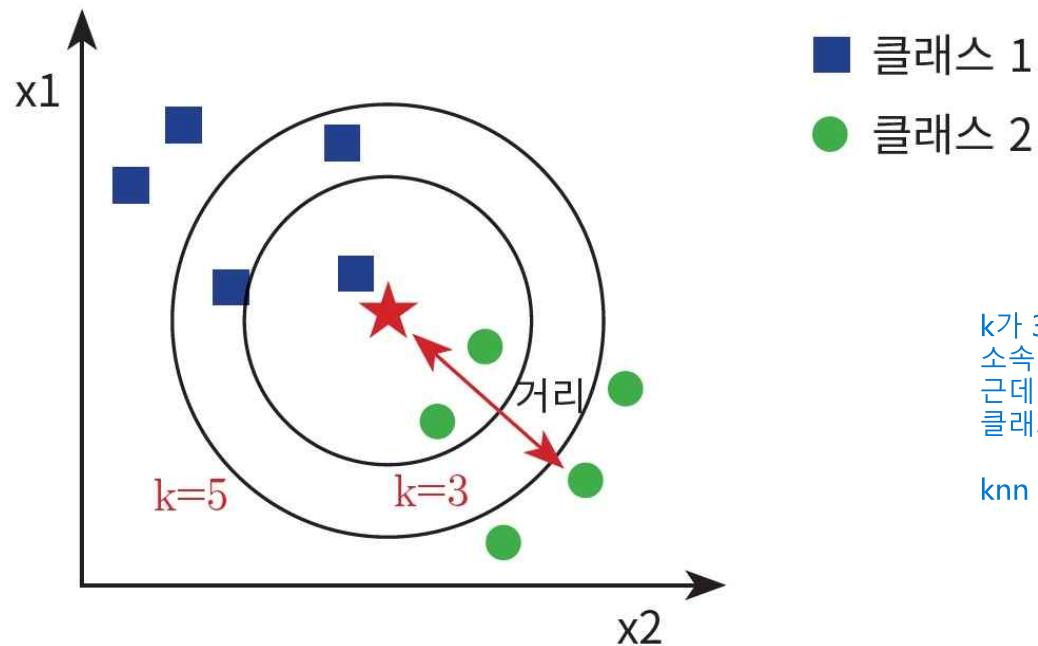
- k-Nearest Neighbor(kNN) 알고리즘은 분류 알고리즘으로 지도학습
 - 레이블이 정해진 데이터 자체가 학습된 결과임



학습이 따로 없다.
x1, x2 픽처를 갖는다면
픽처값에 따라서 배치만 하면 학습의 결과가 된다.

02 kNN 알고리즘

- 이제 새로운 데이터가 입력되어서 그래프 상에 별표로 표시되었다고 하자. 별표는 파랑색 사각형과 연두색 원 중에서 하나에 속해야 한다. 이것을 분류(classification)라고 한다.



k가 3이면 1대2
소속이 클래스 2가 된다
근데 k가 5명 네모가 3개 동그라미 2개니까
클래스1로 바뀐다

knn = k개의 이웃

$$D \geq \frac{k}{\delta} \quad (k=4)$$

이걸 어떻게 해결해야할까???

거리를 가중치로 쓴다.

kNN 알고리즘

kNN 알고리즘

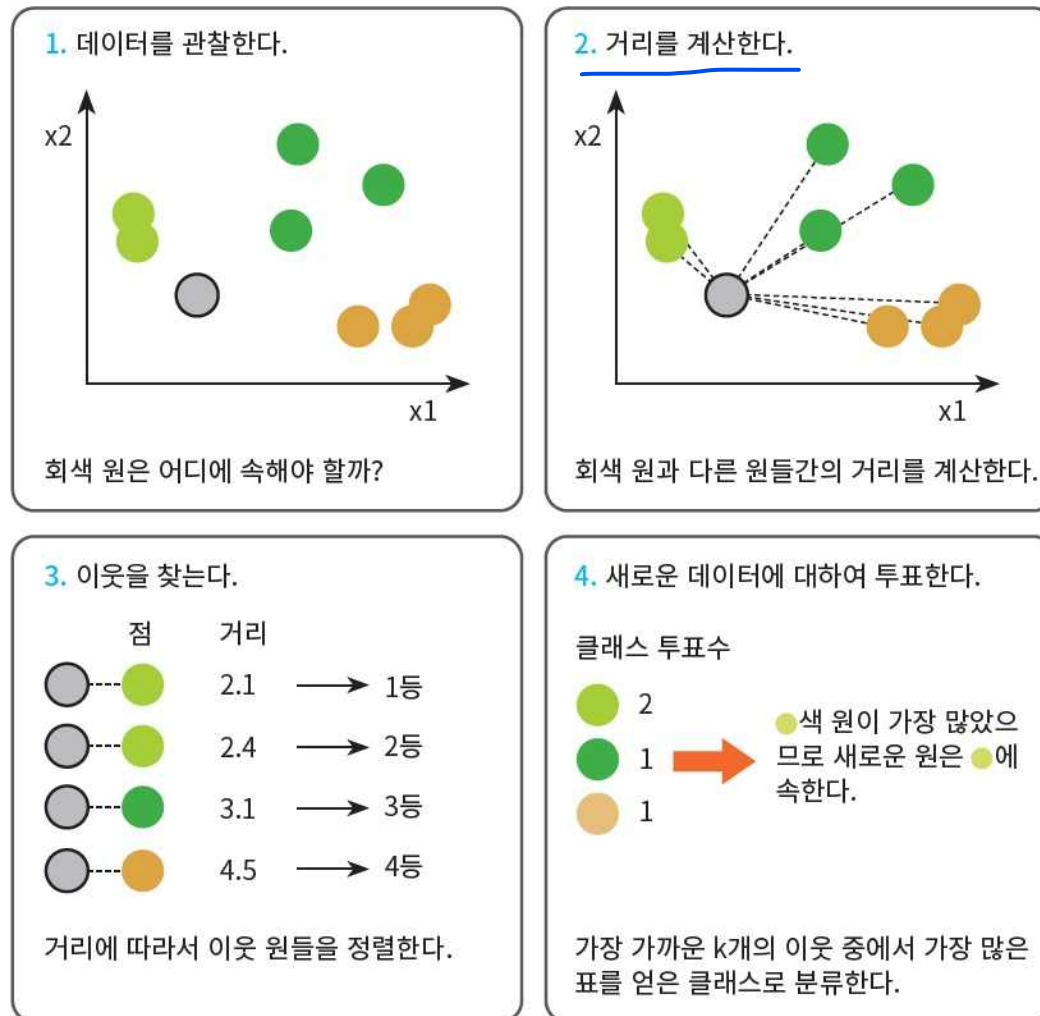


그림 11-3 kNN 알고리즘

kNN 알고리즘

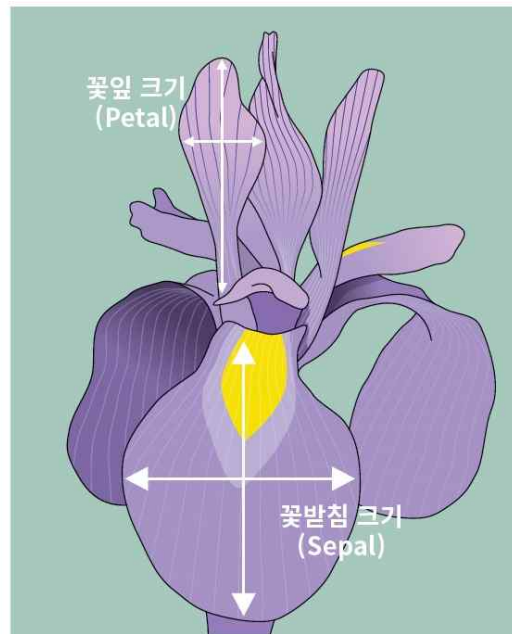
- 수정된 kNN 알고리즘
 - 신입 멤버와의 거리에 따라 각 도형에 약간의 가중치를 준다
 - 신입 멤버와 가까운 도형들은 더 높은 가중치를 얻고, 다른 도형들은 더 낮은 가중치를 갖는다
 - 그런 다음 모든 도형의 가중치를 각 도형별로 전부 합한다
 - 가장 큰 가중치를 얻은 도형이 새로운 멤버의 클래스가 된다

kNN 알고리즘의 장점과 단점

- 장점
 - 어떤 종류의 학습이나 준비 시간이 필요 없다.
- 단점
 - 특징 공간에 있는 모든 데이터에 대한 정보가 필요하다. 왜냐하면, 가장 가까운 이웃을 찾기 위해 새로운 데이터에서 모든 기존 데이터까지의 거리를 확인해야 하기 때문이다. 데이터와 클래스가 많이 있다면, 많은 메모리 공간과 계산 시간이 필요하다.

03 sklearn을 이용한 kNN 알고리즘 실습

- sklearn 라이브러리에는 아이리스 데이터 세트가 있다. 아이리스 데이터 세트에는 150개의 아이리스 꽃 샘플이 있다.
 - 꽃받침(sepal) 길이, 너비, 꽃잎(petal) 길이, 너비, 꽃 이름(label)



```
5.1,3.8,1.9,0.4,Iris-setosa
4.8,3.0,1.4,0.3,Iris-setosa
5.1,3.8,1.6,0.2,Iris-setosa
4.6,3.2,1.4,0.2,Iris-setosa
5.3,3.7,1.5,0.2,Iris-setosa
5.0,3.3,1.4,0.2,Iris-setosa
7.0,3.2,4.7,1.4,Iris-versicolor
6.4,3.2,4.5,1.5,Iris-versicolor
6.9,3.1,4.9,1.5,Iris-versicolor
5.5,2.3,4.0,1.3,Iris-versicolor
6.5,2.8,4.6,1.5,Iris-versicolor
5.7,2.8,4.5,1.3,Iris-versicolor
```

(아이리스 꽃: 꽃받침 길이와 너비, 꽃잎 길이와 너비)

sklearn을 이용한 kNN 알고리즘 실습

```
from sklearn.datasets import load_iris
iris = load_iris()
print(iris.data)
```

```
array([[5.1, 3.5, 1.4, 0.2],
       [4.9, 3. , 1.4, 0.2],
       [4.7, 3.2, 1.3, 0.2],
       [4.6, 3.1, 1.5, 0.2],
       [5. , 3.6, 1.4, 0.2],
       [5.4, 3.9, 1.7, 0.4],
       ...])
```

아이리스 데이터 세트

[illegible]

sklearn을 이용한 kNN 알고리즘 실습

```
# 4개의 특징 이름을 출력한다.  
print(iris.feature_names)
```

```
['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
```

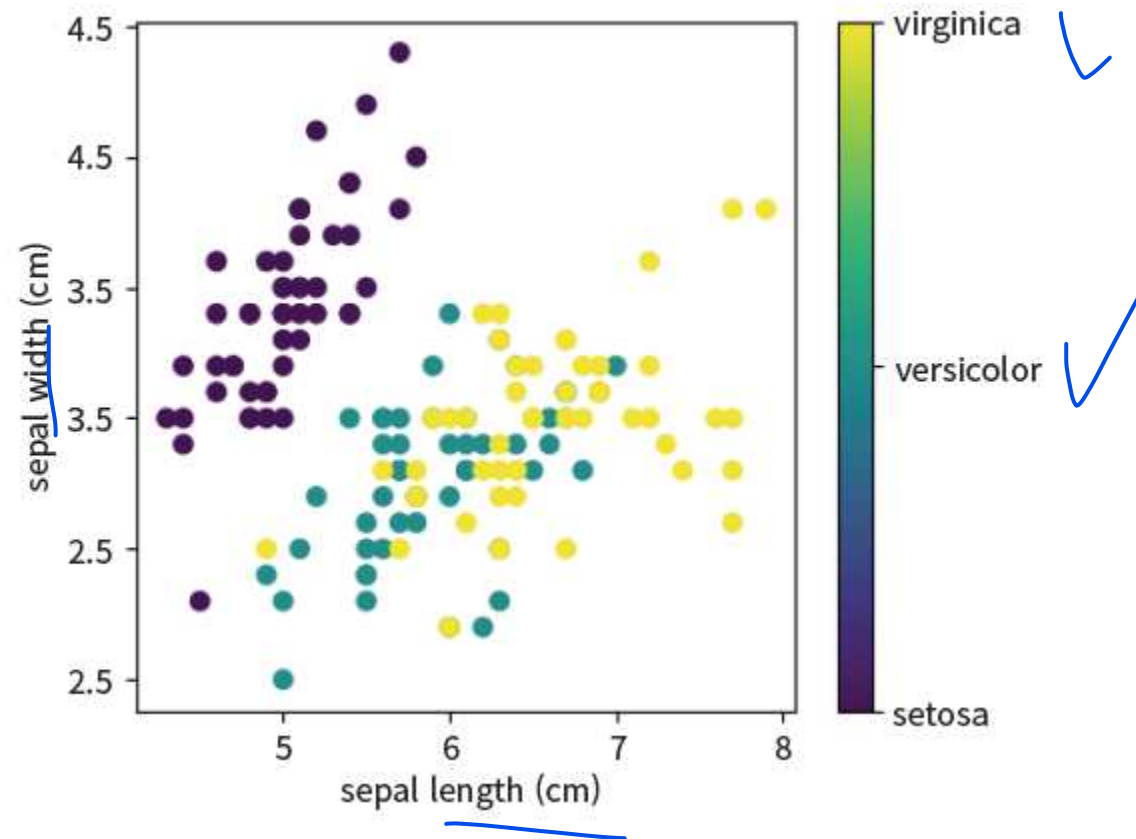
```
# 정수는 꽃의 종류를 나타낸다.: 0 = setosa, 1=versicolor, 2=virginica  
print(iris.target)
```

```
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0  
0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1  
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2  
2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2  
2 2]
```

그래프 그리기

4차원으로 그릴 수 없으니
특징 3개만 함

꽃바침 너비, 길이



두개 구분이 잘 안감

kNN학습

```
from sklearn.model_selection import train_test_split

X = iris.data
y = iris.target

# (80:20)으로 분할한다.
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=4)

print(X_train.shape)
print(X_test.shape)
```

```
(120, 4)
(30, 4)
```

kNN학습

```
from sklearn.neighbors import KNeighborsClassifier  
from sklearn import metrics
```

```
knn = KNeighborsClassifier(n_neighbors=6)  $K=6$   
knn.fit(X_train, y_train) 훈련
```

```
y_pred = knn.predict(X_test) 예측  
scores = metrics.accuracy_score(y_test, y_pred)
```

정답 $\uparrow$ 예측

0.9666666666666667

96%

kNN예측 KNN2.py

- 데이터를 나누지 않고 모두 훈련에 사용하고 새 데이터로 테스트하는 경우

```
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X, y)

#0 = setosa, 1=versicolor, 2=virginica
classes = {0:'setosa',1:'versicolor',2:'virginica'}

# 아직 보지 못한 새로운 데이터를 제시해보자.
x_new = [[3,4,5,2],   가상 데이터
         [5,4,2,2]]
y_predict = knn.predict(x_new)

print(classes[y_predict[0]])
print(classes[y_predict[1]])
```

```
versicolor ✓
setosa ✓
```

예제: kNN으로 필기체 이미지 분류

- sklearn을 사용하여 필기체 숫자 이미지를 인식하는 프로그램을 작성해보자.

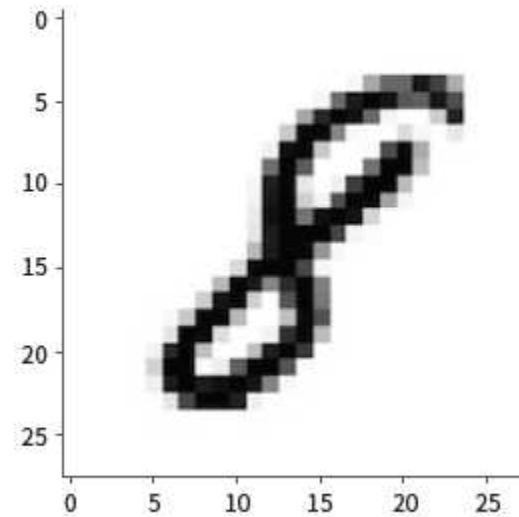
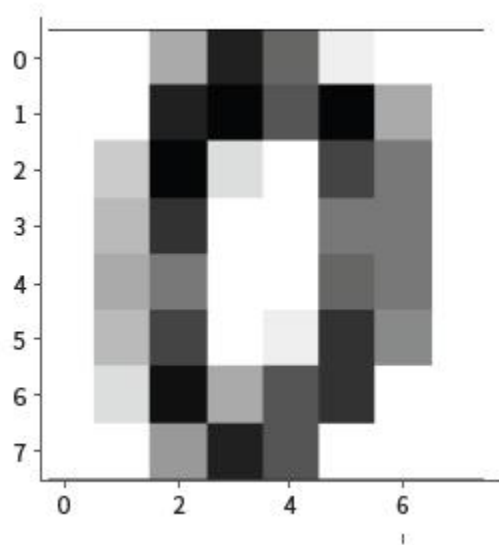


그림 11-6 MNIST 데이터

이미지 표시하기

```
import matplotlib.pyplot as plt
from sklearn import datasets, metrics
from sklearn.model_selection import train_test_split
digits = datasets.load_digits()
plt.imshow(digits.images[0], cmap=plt.cm.gray_r, interpolation='nearest')
```



2차원 이미지를 1차원 배열로 변경

```
n_samples = len(digits.images)  
data = digits.images.reshape((n_samples, -1))
```

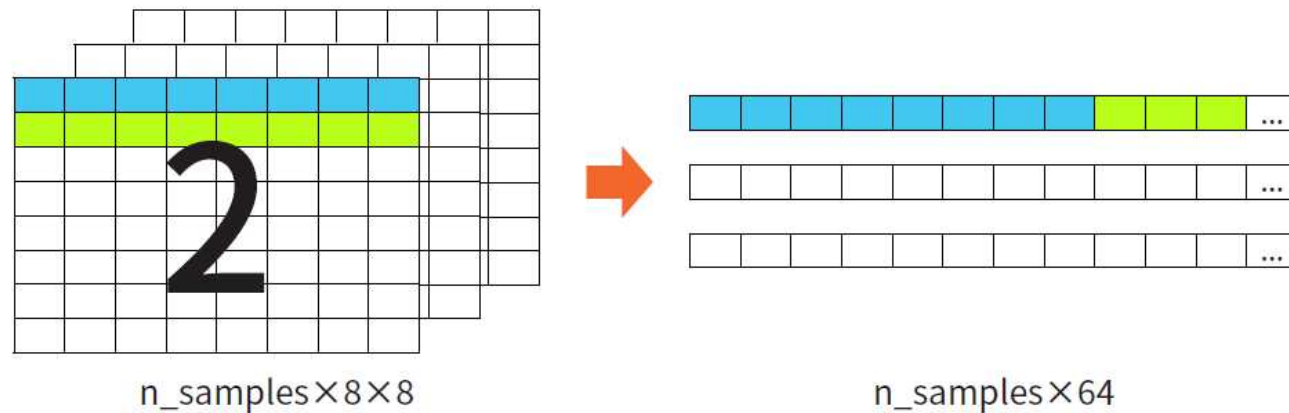


그림 11-7 평탄화

학습 데이터와 테스트 데이터 분리

```
X_train, X_test, y_train, y_test = train_test_split(
    data, digits.target, test_size=0.2)

from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors=6)

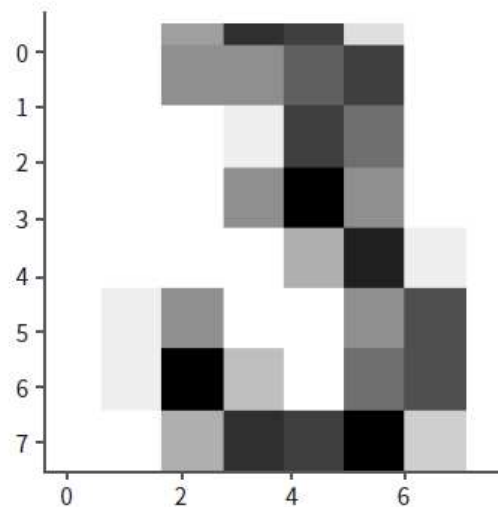
knn.fit(X_train, y_train)
# 테스트 데이터로 예측해 본다.
y_pred = knn.predict(X_test)
# 정확도를 계산한다.
scores = metrics.accuracy_score(y_test, y_pred)
print(scores)
```

0.9532814238042269

테스트

```
# 이미지를 출력하기 위하여 평탄화된 이미지를 다시 8×8 형상으로 만든다.  
plt.imshow(X_test[10].reshape(8,8), cmap=plt.cm.gray_r, interpolation='nearest')  
y_pred = knn.predict([X_test[10]]) # 입력은 항상 2차원 행렬이어야 한다.  
print(y_pred)
```

[3]



머신 러닝의 성능평가

- **sklearn** 라이브러리에는 혼동 행렬과 분류 리포트를 출력하는 모듈이 포함되어 있다

		Predicted Class	
True Class		True	False
	True	True Positive(TP)	False Negative(FN)
	False	False Positive(FN)	True Negative(TN)

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F1-score = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

혼동 행렬 출력

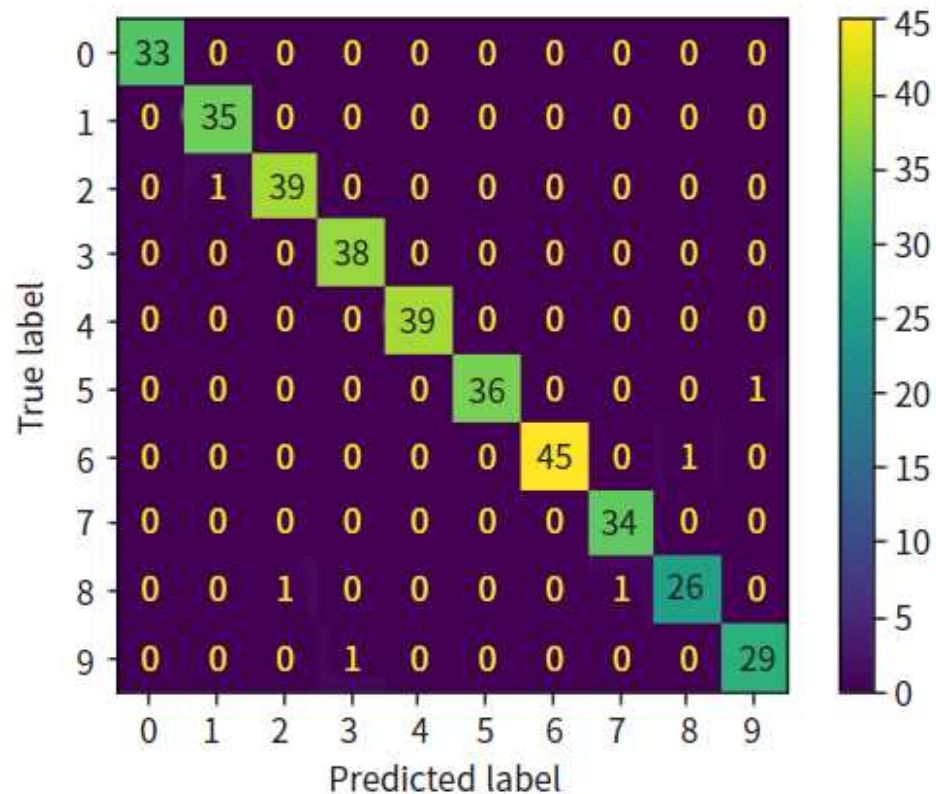
```
import matplotlib.pyplot as plt
from sklearn import datasets, metrics
from sklearn.model_selection import train_test_split

digits = datasets.load_digits()
n_samples = len(digits.images)
data = digits.images.reshape((n_samples, -1))

from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors=6)

X_train, X_test, y_train, y_test = train_test_split(
    data, digits.target, test_size=0.2)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)
disp = metrics.plot_confusion_matrix(knn, X_test, y_test)
plt.show()
```

후동 행렬 출력



결과를 보면 클래스 0에 속하는 샘플 33은 모두 올바르게 분류하고 있음을 알 수 있다. 클래스 2는 총 40개인데 1개는 잘못 분류하였고 나머지 39개는 올바르게 분류하고 있다.

분류 리포트 출력

```
print(f"{metrics.classification_report(y_test, y_pred)}\n")
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	28
1	0.95	1.00	0.97	39
2	1.00	1.00	1.00	33
3	0.95	1.00	0.97	36
4	1.00	1.00	1.00	35
5	1.00	1.00	1.00	35
6	1.00	1.00	1.00	45
7	1.00	1.00	1.00	40
8	0.97	0.93	0.95	40
9	1.00	0.93	0.96	29
accuracy			0.99	360
macro avg	0.99	0.99	0.99	360
weighted avg	0.99	0.99	0.99	360



kNN 회귀 알고리즘

농어의 무게를 예측하라!

길이를 가지고 무게를 예측

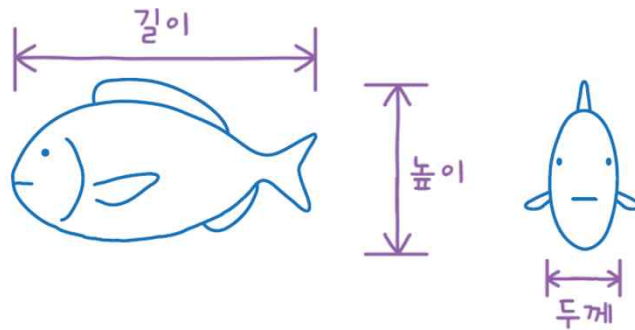
학습목표

- 지도 학습 알고리즘의 한 종류인 kNN 회귀 알고리즘에 대해 배웁니다.

SECTION 11-1 k-최근접 이웃 회귀(1)

- 농어 샘플 56개의 높이, 길이 등 수치로 무게를 예측하기
- 지도 학습 알고리즘은 크게 분류와 회귀(regression)로 나뉨
 - 분류: 샘플을 몇 개의 클래스 중 하나로 분류
 - 회귀: 클래스 중 하나로 분류하는 것이 아니라 임의의 어떤 숫자를 예측
 - ⑩ 예) 내년도 경제 성장률을 예측하거나 배달이 도착할 시간을 예측
 - ⑩ 회귀는 정해진 클래스가 없고 임의의 수치를 출력

연속된 값을 예측 하는 게 회귀

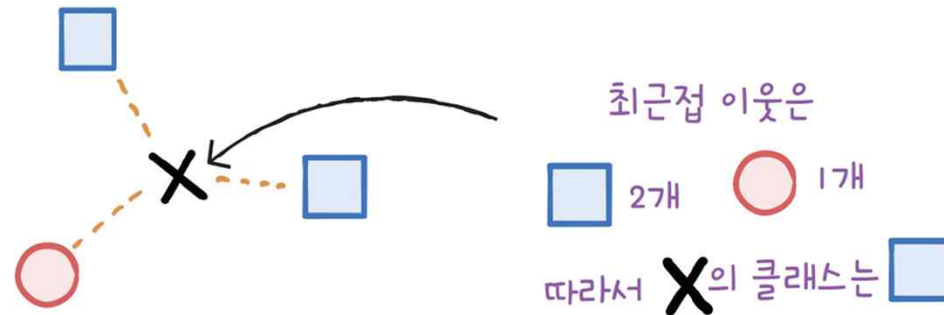


SECTION 11-1 k-최근접 이웃 회귀(2)

▪ k-최근접 이웃 분류 알고리즘

- ⑩ 예측하려는 샘플에 가장 가까운 샘플 k개를 선택
- ⑩ 이 샘플들의 클래스를 확인하여 다수 클래스를 새로운 샘플의 클래스로 예측
- ⑩ 다음 그림의 왼쪽에 k-최근접 이웃 분류가 잘 나타남
- ⑩ $k = 3$ (샘플이 3개)이라 가정하면 사각형이 2개로 다수이기 때문에 새로운 샘플의 클래스는 사각형

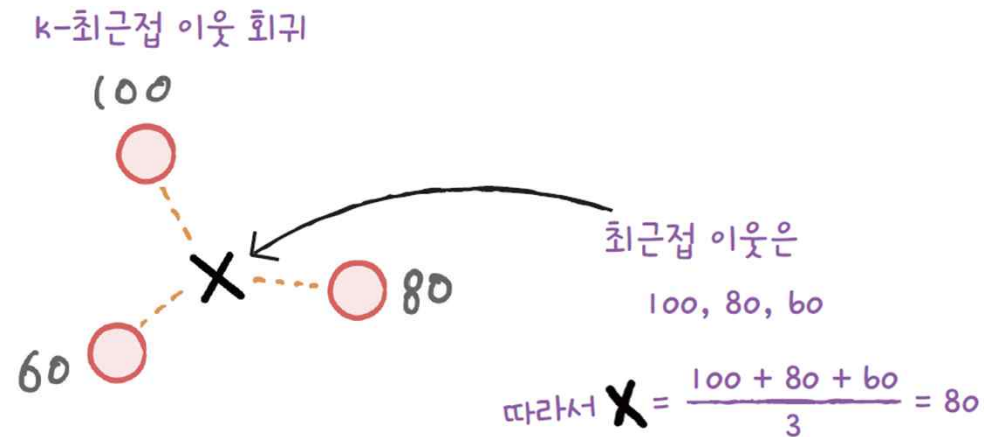
k-최근접 이웃 분류



SECTION 11-1 k-최근접 이웃 회귀(3)

▪ k-최근접 이웃 회귀 알고리즘

- ⑩ 분류와 똑같이 예측하려는 샘플에 가장 가까운 샘플 k개를 선택
- ⑩ 회귀이기 때문에 이웃한 샘플의 타깃은 어떤 클래스가 아니라 임의의 수치
- ⑩ 이웃 샘플의 수치를 사용해 새로운 샘플의 타깃을 예측하기 위해 이 수치들의 평균을 구함
- ⑩ 그림에서 이웃한 샘플의 타깃값이 각각 100, 80, 60이고 이를 평균하면 샘플의 예측 타깃값은 80



SECTION 11-1 k-최근접 이웃 회귀(4)

◦ 데이터 준비: <https://github.com/rickiepark/hg-mldl>

▪ 훈련 데이터 준비

⑩ 농어의 길이만 있어도 무게를 잘 예측할 수 있다고 가정 (농어의 길이가 특성, 무게가 타겟)

⑩ 넘파이 배열로 만들기 (소스: http://bit.ly/perch_data에서 복사)

```
import numpy as np
perch_length = np.array(
    [ 8.4, 13.7, 15.0, 16.2, 17.4, 18.0, 18.7, 19.0, 19.6, 20.0,
      21.0, 21.0, 21.0, 21.3, 22.0, 22.0, 22.0, 22.0, 22.0, 22.5,
      22.5, 22.7, 23.0, 23.5, 24.0, 24.0, 24.6, 25.0, 25.6, 26.5,
      27.3, 27.5, 27.5, 27.5, 28.0, 28.7, 30.0, 32.8, 34.5, 35.0,
      36.5, 36.0, 37.0, 37.0, 39.0, 39.0, 39.0, 40.0, 40.0, 40.0,
      40.0, 42.0, 43.0, 43.0, 43.5, 44.0]
)
perch_weight = np.array(
    [ 5.9, 32.0, 40.0, 51.5, 70.0, 100.0, 78.0, 80.0, 85.0, 85.0,
      110.0, 115.0, 125.0, 130.0, 120.0, 120.0, 130.0, 135.0, 110.0,
      130.0, 150.0, 145.0, 150.0, 170.0, 225.0, 145.0, 188.0, 180.0,
      197.0, 218.0, 300.0, 260.0, 265.0, 250.0, 250.0, 300.0, 320.0,
      514.0, 556.0, 840.0, 685.0, 700.0, 700.0, 690.0, 900.0, 650.0,
      820.0, 850.0, 900.0, 1015.0, 820.0, 1100.0, 1000.0, 1100.0,
      1000.0, 1000.0]
)
```

SECTION 11-1 k-최근접 이웃 회귀(5)

◦ 데이터 준비

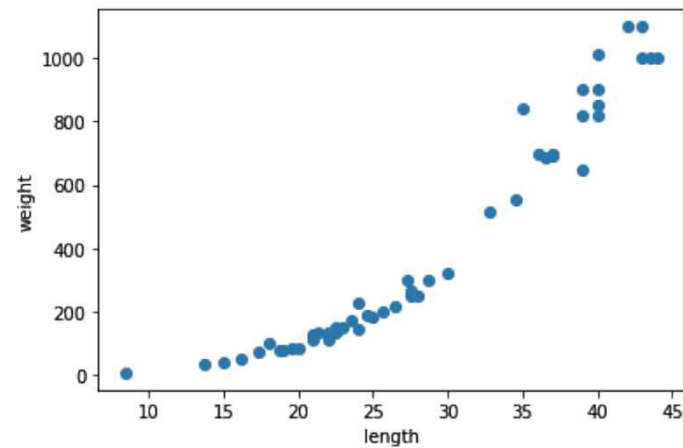
▪ 훈련 데이터 준비

⑩ 데이터의 형태 파악을 위해 산점도 그리기

⑩ 하나의 특성을 사용하기 때문에 특성 데이터를 x축, 타깃 데이터를 y축

⑩ 맷플롯립을 임포트하고 scatter() 함수를 사용하여 산점도 그리기

```
import matplotlib.pyplot as plt
plt.scatter(perch_length, perch_weight)
plt.xlabel('length')
plt.ylabel('weight')
plt.show()
```



SECTION 11-1 k-최근접 이웃 회귀(6)

◦ 데이터 준비

▪ 훈련 세트와 테스트 세트로 나누기

⑩ 사이킷런의 `train_test_split()` 함수를 사용

⑩ 책과 결과를 동일하게 유지하기 위해 `random_state=42`로 지정

```
from sklearn.model_selection import train_test_split
train_input, test_input, train_target, test_target = train_test_split(
    perch_length, perch_weight, random_state=42)
```

⑩ 사이킷런에 사용할 훈련 세트는 2차원 배열이어야 함

⑩ `perch_length`가 1차원 배열이기 때문에 이를 나눈 `train_input`과 `test_input`도 1차원 배열

- 이런 1차원 배열을 1개의 열이 있는 2차원 배열로 바꿔야 함

`[1, 2, 3]` → `[[1], [2], [3]]`

크기 : (3,) 크기 : (3, 1)

⑩ 파이썬에서 1차원 배열의 크기는 원소가 1개인 튜플

- 예를 들어 `[1, 2, 3]`의 크기는 (3,)

- 이를 2차원 배열로 만들기 위해 억지로 하나의 열을 추가하여 배열의 크기가 (3, 1)이 됨

- 배열을 나타내는 방식만 달라졌을 뿐 배열에 있는 원소의 개수는 동일하게 3개

SECTION 11-1 k-최근접 이웃 회귀(7)

◦ 데이터 준비

▪ 훈련 세트와 테스트 세트로 나누기

⑩ 크기를 바꿀 수 있는 reshape() 메서드

- 2장에서는 2개의 특성을 사용했기 때문에 자연스럽게 열이 2개인 2차원 배열을 사용
- 이번 예제에서는 특성을 1개만 사용하므로 수동으로 2차원 배열을 만들어 줌

⑩ (4,) 배열을 (2, 2) 크기로 바꾸기

```
test_array = np.array([1,2,3,4])  
print(test_array.shape)
```

→ (4,)



```
test_array = test_array.reshape(2, 2)  
print(test_array.shape)
```

→ (2, 2)



지정한 크기와 원본 배열의 원소 개수가 달라도 되나?

- reshape() 메서드는 크기가 바뀐 새로운 배열을 반환할 때 지정한 크기가 원본 배열에 있는 원소의 개수와 다르면 에러가 발생
- 예를 들어 다음과 같이 (4,) 크기의 배열을 (2, 3)으로 바꾸려고 하면 원본 배열의 원소는 4개인데 $2 \times 3 = 6$ 개로 바꾸려고 하기 때문에 에러 발생

SECTION 11-1 k-최근접 이웃 회귀(8)

◦ 데이터 준비

▪ 훈련 세트와 테스트 세트로 나누기

⑩ reshape() 메서드를 사용해 train_input과 test_input을 2차원 배열로 변환

- train_input의 크기는 (42,)

- 이를 2차원 배열인 (42, 1)로 바꾸려면 train_input.reshape(42, 1)과 같이 사용

⑩ 넘파이의 배열의 크기를 자동으로 지정하는 기능

- 크기에 -1을 지정하면 나머지 원소 개수로 모두 채우라는 의미

- 첫 번째 크기를 나머지 원소 개수로 채우고, 두 번째 크기를 1로 하려면 train_input.reshape(-1, 1)처럼 사용

⑩ reshape() 메서드로 배열의 크기 변경하기

```
train_input = train_input.reshape(-1, 1)
test_input = test_input.reshape(-1, 1)
print(train_input.shape, test_input.shape)
```



(42, 1) (14, 1)

SECTION 11-1 k-최근접 이웃 회귀(9)

○ 결정계수(R^2)

- 사이킷런에서 k-최근접 이웃 회귀 알고리즘을 구현한 클래스는 KNeighborsRegressor
- 클래스의 사용법은 KNeighborsClassifier와 매우 비슷함
- 객체를 생성하고 fit() 메서드로 회귀 모델을 훈련하기

```
from sklearn.neighbors import KNeighborsRegressor
```

```
knr = KNeighborsRegressor()
```

```
# k-최근접 이웃 회귀 모델을 훈련합니다
```

```
knr.fit(train_input, train_target)
```

```
print(knr.score(test_input, test_target))
```

→ 0.9928094061010639

- 테스트 세트의 점수는 분류에서는 정확도, 회귀에서는 결정계수(R^2)

⑩ 만약 타깃의 평균 정도를 예측하는 수준이라면 (즉, 분자와 분모가 비슷해져) R^2 는 0에 가까워지고, 예측이 타깃에 아주 가까워지면 (분자가 0에 가까워지기 때문에) 1에 가까운 값이 됨

실종연 | ~ |

$$R^2 = 1 - \frac{(\text{타깃} - \text{예측})^2 \text{의 합}}{(\text{타깃} - \text{평균})^2 \text{의 합}}$$

~ |

- NaN |

SECTION 11-1 k-최근접 이웃 회귀(10)

◦ 결정계수(R^2)

- 타겟과 예측한 값 사이의 차이를 구하여 어느 정도 예측이 벗어났는지 평가
- 사이킷런 sklearn.metrics 패키지의 mean_absolute_error: 타겟과 예측의 절댓값 오차를 평균하여 반환

```
from sklearn.metrics import mean_absolute_error
```

```
# 테스트 세트에 대한 예측을 만듭니다
```

```
test_prediction = knr.predict(test_input)
```

```
# 테스트 세트에 대한 평균 절댓값 오차를 계산합니다
```

```
mae = mean_absolute_error(test_target, test_prediction)
```

```
print(mae)
```

정답하고 예측값 뺀 거 제공.

평균 제공한 거

→ 19.157142857142862

⑩ 결과에서 예측이 평균적으로 19 gram 정도 타겟값과 차이가 나타남

SECTION 11-1 k-최근접 이웃 회귀(11)

◦ 과대적합 vs 과소적합

- 앞에서 훈련한 모델을 사용해 훈련 세트의 R^2 점수를 확인

```
print(knr.score(train_input, train_target)) → 0.9698823289099255
```

- 모델을 훈련 세트와 테스트 세트에서 평가하면 두 값 중 보통 훈련 세트의 점수가 조금 더 높게 나옴

- 과대적합(overfitting)

⑩ 훈련 세트에서 점수가 굉장히 좋았는데 테스트 세트에서는 점수가 굉장히 나쁜 경우

⑩ 즉, 훈련 세트에만 잘 맞는 모델이라 테스트 세트와 나중에 실전에 투입하여 새로운 샘플에 대한 예측을 만들 때 잘 동작하지 않을 것임

- 과소적합(underfitting)

⑩ 반대로 훈련 세트보다 테스트 세트의 점수가 높거나 두 점수가 모두 너무 낮은 경우

⑩ 즉, 모델이 너무 단순하여 훈련 세트에 적절히 훈련되지 않은 경우



SECTION 11-1 k-최근접 이웃 회귀(12)

◦ 과대적합 vs 과소적합

- 앞의 k-최근접 이웃 회귀로 평가한 훈련 세트와 테스트 세트의 점수는 훈련 세트보다 테스트 세트의 점수가 높은 과소적합
- 문제 해결
 - ⑩ 모델을 조금 더 복잡하게(즉, 훈련 세트에 더 잘 맞게) 만들면 테스트 세트의 점수는 조금 낮아질 것임
 - ⑩ 이웃의 개수 k 를 줄여 k-최근접 이웃 알고리즘으로 모델을 더 복잡하게 만들기
 - ⑩ 이웃의 개수를 줄이면 훈련 세트에 있는 국지적인 패턴에 민감해지고, 이웃의 개수를 늘리면 데이터 전반에 있는 일반적인 패턴을 따르게 됨

SECTION 11-1 k-최근접 이웃 회귀(13)

◦ 과대적합 vs 과소적합

- 사이킷런의 k-최근접 이웃 알고리즘의 기본 k 값인 5를 3으로 낮추기

⑩ n_neighbors 속성값 변경

```
# 이웃의 개수를 3으로 설정합니다
knr.n_neighbors = 3

# 모델을 다시 훈련합니다
knr.fit(train_input, train_target)
print(knr.score(train_input, train_target))
```

→ 0.9804899950518966

- 테스트 세트의 점수 확인

```
print(knr.score(test_input, test_target))
```

→ 0.974645996398761

SECTION 11-1 k-최근접 이웃 회귀(14)

◦ 회귀 문제 다루기(문제해결 과정)

▪ 문제

- ⑩ 농어의 높이, 길이 등의 수치로 무게를 예측하기(회귀는 임의의 수치를 예측)

▪ 해결

- ⑩ k-최근접 이웃 회귀 모델은 분류와 동일하게 가장 먼저 가까운 k개의 이웃을 찾아 이웃 샘플의 타깃값을 평균하여 이 샘플의 예측값으로 사용
- ⑩ 사이킷런은 회귀 모델의 점수로 R^2 , 즉 결정계수 값을 반환. 이 값은 1에 가까울수록 좋음
- ⑩ 정량적인 평가는 사이킷런에서 제공하는 다른 평가 도구를 사용할 수 있음(대표적으로 절댓값 오차)
- ⑩ 모델을 훈련하고 나서 훈련 세트와 테스트 세트에 대해 모두 평가 점수를 구할 수 있음
 - 훈련 세트의 점수와 테스트 세트의 점수 차이가 크면 좋지 않음
 - 일반적으로 훈련 세트의 점수가 테스트 세트보다 조금 더 높음
- ⑩ 과대적합: 만약 테스트 세트의 점수가 너무 낮다면 모델이 훈련 세트에 과도하게 맞춰짐
- ⑩ 과소적합: 테스트 세트 점수가 너무 높거나 두 점수가 모두 낮은 경우
- ⑩ 과대적합일 경우 모델을 덜 복잡하게 만들어야 함(k-최근접 이웃의 경우 k 값을 늘림)
- ⑩ 과소적합일 경우 모델을 더 복잡하게 만들어 줌(k-최근접 이웃의 경우 k 값을 줄임)

SECTION 11-1 마무리(1)

- 키워드로 끝나는 핵심 포인트

- 회귀는 임의의 수치를 예측하는 문제. 따라서 타깃값도 임의의 수치
- k-최근접 이웃 회귀는 k-최근접 이웃 알고리즘을 사용해 회귀 문제 해결
 - ⑩ 가장 가까운 이웃 샘플을 찾고 이 샘플들의 타깃값을 평균하여 예측
- 결정계수(R^2)는 대표적인 회귀 문제의 성능 측정 도구
 - ⑩ 1에 가까울수록 좋고, 0에 가깝다면 성능이 나쁜 모델
- 과대적합은 모델의 훈련 세트 성능이 테스트 세트 성능보다 훨씬 높을 때 발생
 - ⑩ 모델이 훈련 세트에 너무 집착해서 데이터에 내재된 거시적인 패턴을 감지하지 못함
 - ⑩ 과소적합은 이와 반대로 훈련 세트와 테스트 세트 성능이 모두 동일하게 낮거나 테스트 세트 성능이 오히려 더 높을 때 발생. 이런 경우 더 복잡한 모델을 사용해 훈련 세트에 잘 맞는 모델을 만들어야 함

SECTION 11-1 마무리(2)

- 핵심 패키지와 함수

- scikit-learn

- ⑩ KNeighborsRegressor: k-최근접 이웃 회귀 모델을 만드는 사이킷런 클래스

- ⑩ mean_absolute_error(): 회귀 모델의 평균 절댓값 오차를 계산

- numpy

- ⑩ reshape(): 배열의 크기를 바꾸는 메서드

SECTION 11-1 확인 문제

앞서 만든 k-최근접 이웃 회귀 모델의 k 값을 1, 5, 10으로 바꿔가며 훈련하고, 농어의 길이를 5에서 45까지 바꿔가며 예측을 만들어 그래프로 나타내기. n이 커짐에 따라 모델이 단순해지는 것을 볼 수 있는가? [노트] 맷플롯립의 plot() 함수는 x축과 y축의 값을 받아 선 그래프를 그려줌

```
# k-최근접 이웃 회귀 객체를 만듭니다
knr = KNeighborsRegressor()
# 5에서 45까지 x 좌표를 만듭니다
x = np.arange(5, 45).reshape(-1, 1)
# n = 1, 5, 10일 때 예측 결과를 그래프로 그립니다
for n in [1, 5, 10]:
    # 모델을 훈련합니다
    knr.n_neighbors =          # 이 라인의 코드를 완성해 보세요
    knr.fit(train_input, train_target)
    # 지정한 범위 x에 대한 예측을 구합니다
    prediction =          # 이 라인의 코드를 완성해 보세요
    # 훈련 세트와 예측 결과를 그래프로 그립니다
    plt.scatter(train_input, train_target)
    plt.plot(x, prediction)
    plt.show()
```

04 비지도 학습(k-means 클러스터링)

- 비지도 학습 중에서 가장 대표적인 것이 K-means 알고리즘이다.

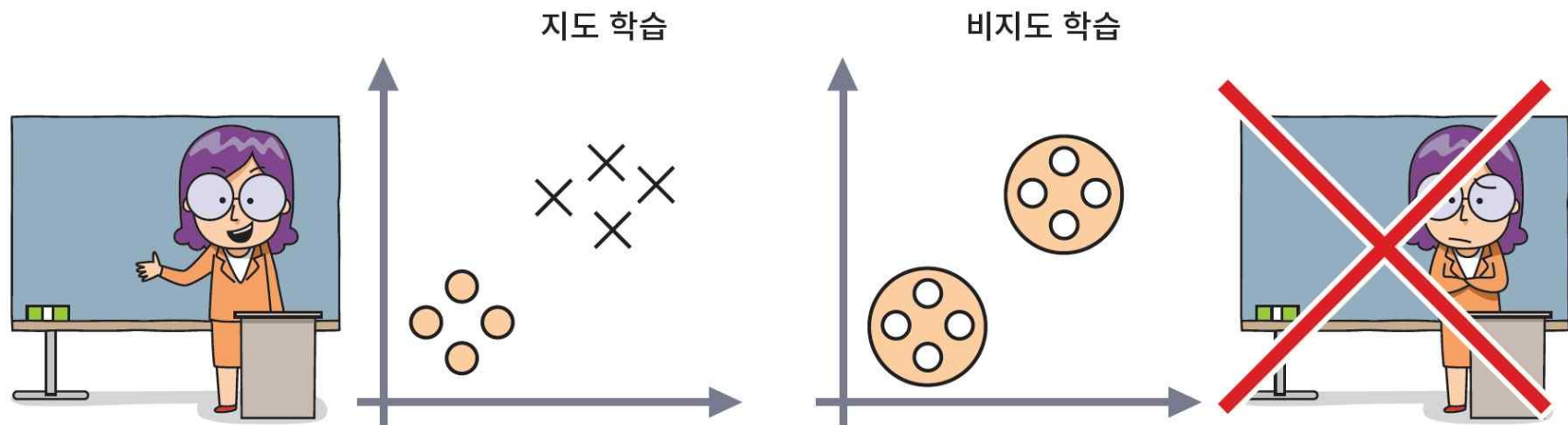
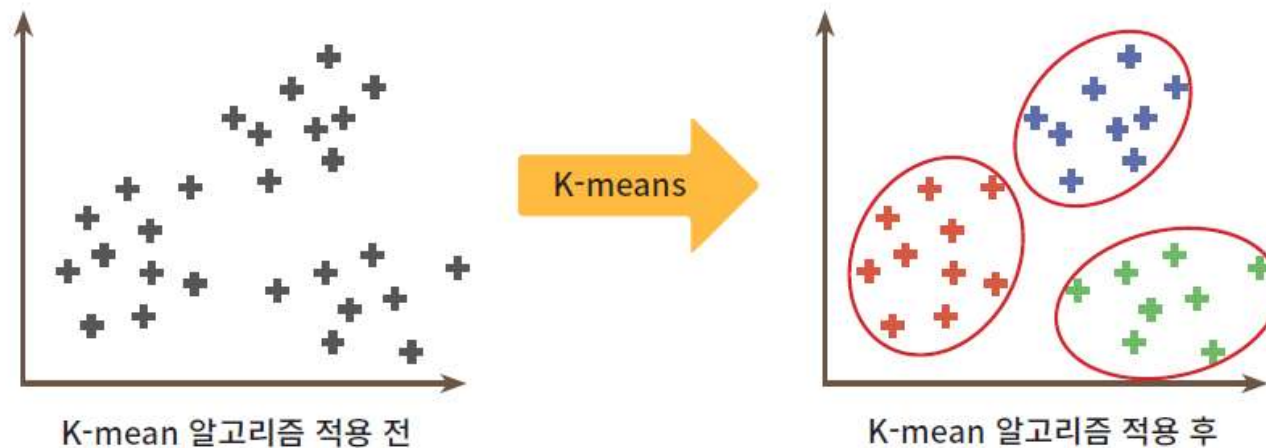


그림 11-6 지도 학습과 비지도 학습

K-means 클러스터링

- K-means 알고리즘(K-means algorithm)은 주어진 n 개의 관측값을 k 개의 클러스터로 분할하는 알고리즘으로, 관측값들은 거리가 최소인 클러스터로 분류된다.
 - 단점: NP-hard로 시간이 많이 걸린다.



- P (Polynomial time)
 - 입력 크기 n 에 대해 다항식 시간 안에 풀 수 있는 문제
 - 정렬, 최단경로(Dijkstra), 최대공약수
- NP (Nondeterministic Polynomial time)
 - 정답이 주어졌을 때 검증은 빠르게(다항시간에) 할 수 있는 문제
 - 지금까지 다항시간 알고리즘이 발견되지 않음
 - 스도쿠, 배낭문제, 여행자문제(TSP)
- NP-hard
 - NP보다 어렵거나 같은 문제
 - 검증조차 다항시간에 불가능할 수도 있음
- NP-complete
 - NP 문제 중에서 가장 어려운 문제들
 - 다른 모든 NP 문제를 이 문제로 변환(reduction)할 수 있음

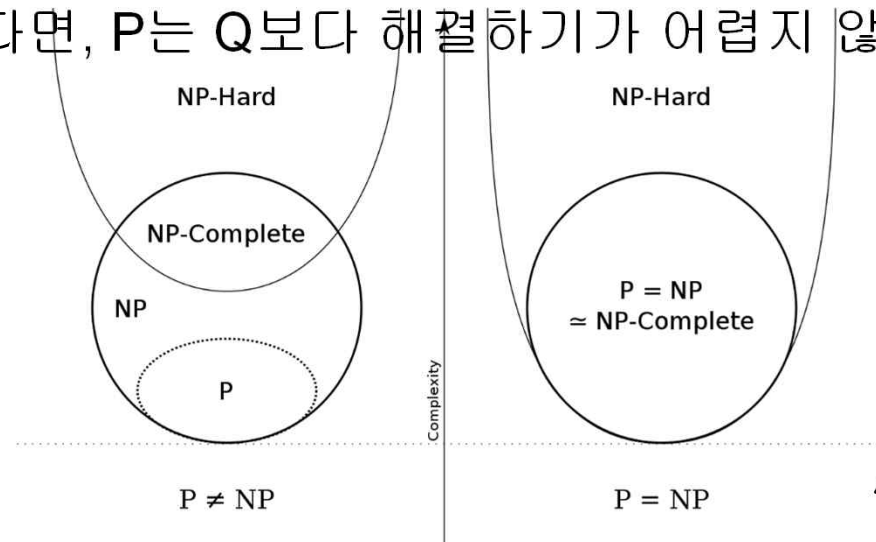
이게 풀리면 모든 np문제가 풀린다.

모든 np문제를 다 풀 수 있는 문

- Class P란 다항시간(**polynomial time**) 내에 풀리는 문제를 가리킴
- Class NP란 운이 좋으면 다항시간에 풀리는(**nondeterministic polynomial**) 문제로, 어떤 답이 해당 문제의 정답인지 여부를 다항시간 내에 확인(**verified**)할 수 있는 문제를 가리킴
- NP-complete란 모든 NP 문제를 해당 문제로 바꿔 풀 수 있을(**reduce**) 때 해당 문제를 가리킴

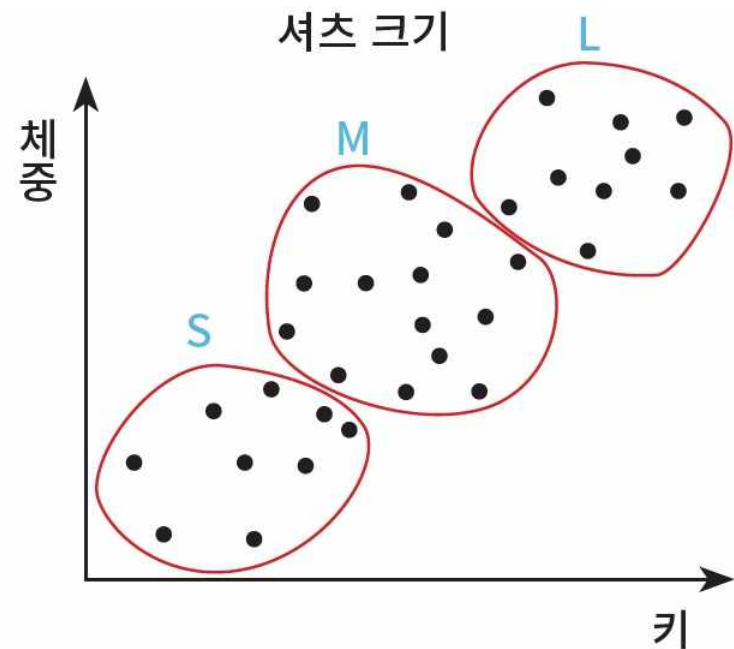
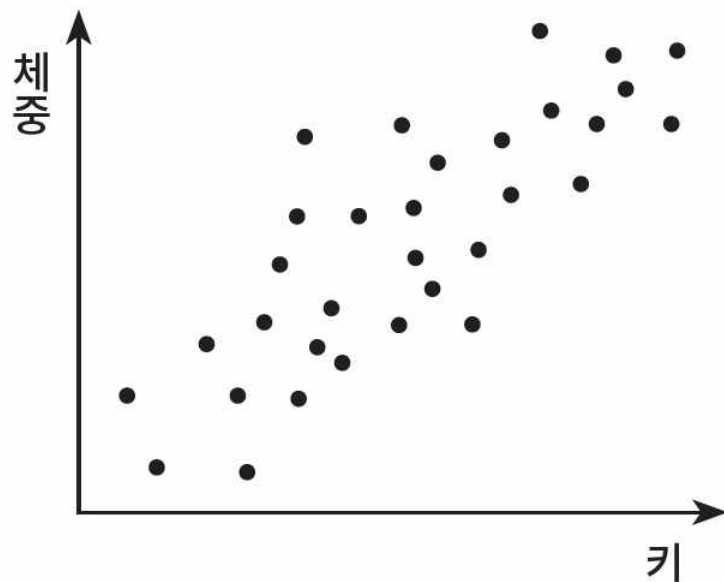
NP-hard이면서 Class NP에 속하는 문제

- Reducibility란 어떤 문제 P를 Q로 바꿔 풀 수 있는 경우를 가리킴. 만일 P가 Q로 **reduce**할 수 있다면, P는 Q보다 해결하기가 어렵지 않다고 말할 수 있음



K-means 클러스터링

- 셔츠를 만들어서 판매하는 회사를 생각해보자. 회사는 시장에 새로운 셔츠 모델을 공개하여야 한다. 회사는 사람들의 키와 체중을 조사하여 그래프로 그려보았다고 하자.



K-means 알고리즘

- 분할법
 - 주어진 데이터를 여러 그룹으로 나누는 방법
- 클러스터를 나누는 과정
 - 거리 기반의 비용 함수(cost function)를 최소화하는 방식
 - 비용 함수: 각 클러스터의 중심(centroid)과 클러스터 내의 데이터와의 거리를 제공한 값
 - 이 함수값을 최소화하는 방향으로 각 데이터의 소속 클러스터를 업데이트 해 줌
 - 같은 클러스터 내 데이터끼리의 유사도는 증가
 - 다른 클러스터에 있는 데이터와의 유사도는 감소

K-means 클러스터링 알고리즘

입력값

1. k : 클러스터 수
2. D : n 개의 데이터

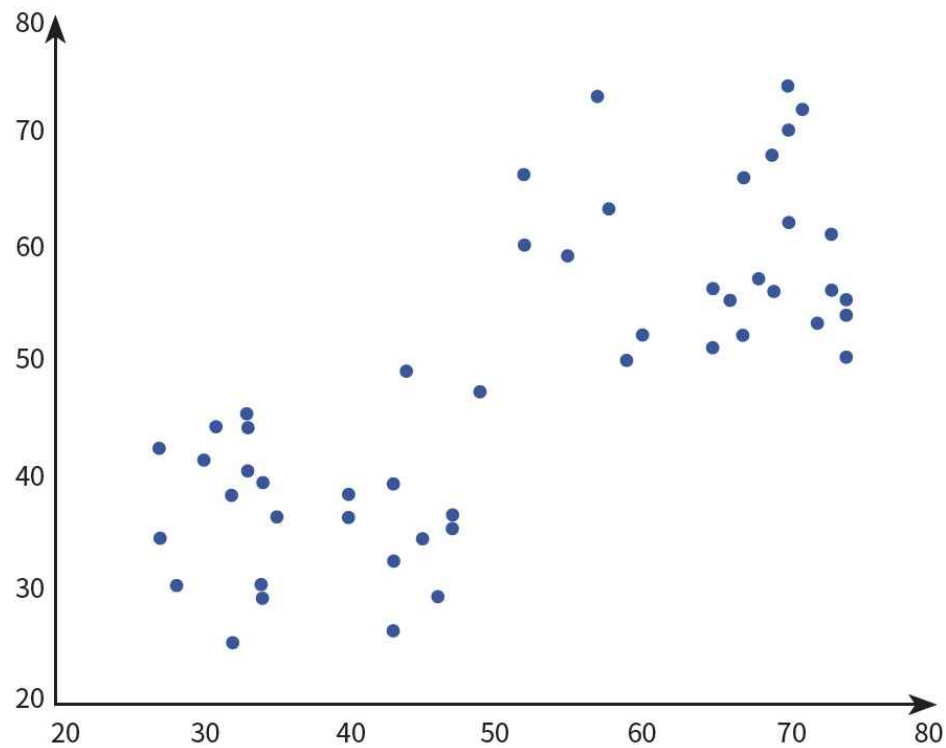
출력값: k 개의 클러스터

알고리즘

1. 집합 D 에서 k 개의 데이터를 임의로 추출하고, 이 데이터들을 각 클러스터의 중심 (centroid) 으로 설정한다. (초기값 설정)
2. 집합 D 의 각 데이터에 대해 k 개의 클러스터 중심과의 거리를 계산하고, 각 데이터가 어느 중심점 (centroid)과 가장 유사도가 높은지 알아낸다. 그리고 그렇게 찾아낸 중심점으로 각 데이터들을 할당한다.
3. 클러스터의 중심점을 다시 계산한다. 즉, 2에서 재할당된 클러스터들을 기준으로 중심점을 다시 계산한다.
4. 각 데이터의 소속 클러스터가 바뀌지 않을 때까지 과정 2, 3을 반복한다.

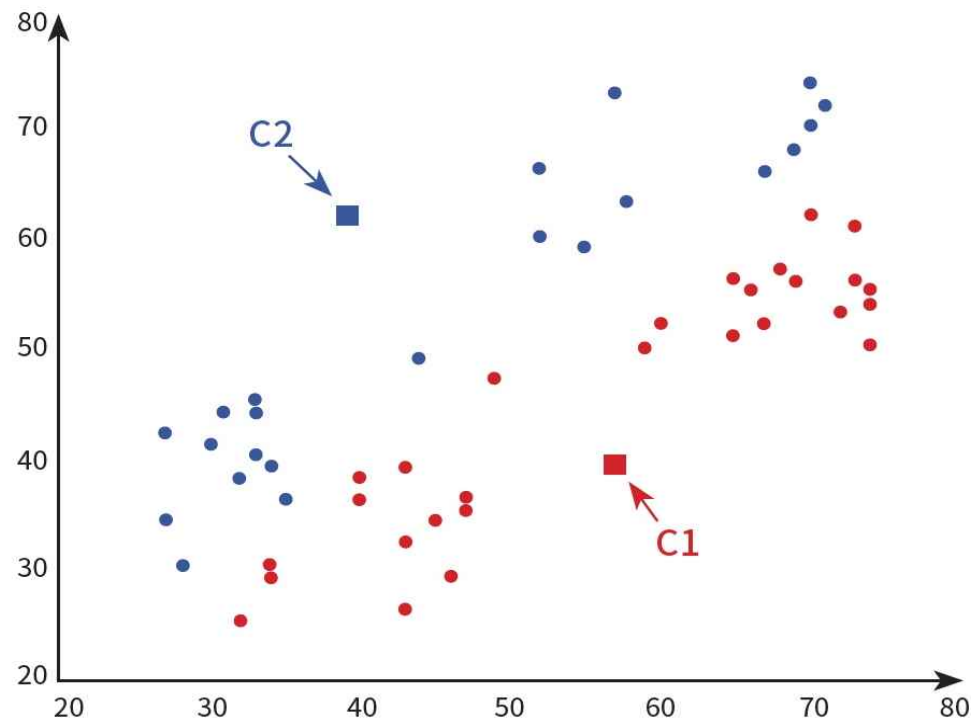
알고리즘 #1

- 알고리즘을 설명하기 위하여 다음과 같이 데이터들이 주어졌다고 하자. 우리는 데이터를 2개의 그룹으로 나누어야 한다. 즉 $k=2$ 이다.



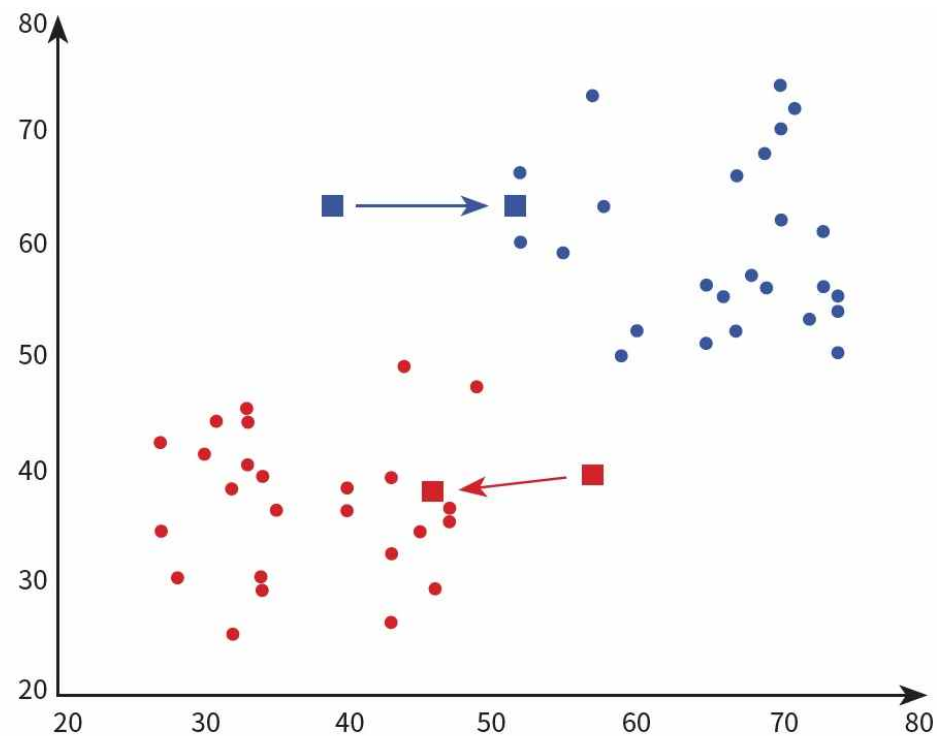
알고리즘 #2

- 알고리즘은 무작위로 2개의 중심점을 선택한다. 이것을 **C1**과 **C2**라고 하자. 각 점에서 중심점까지의 거리를 계산한 후 더 가까운 중심점으로 소속을 바꾼다.



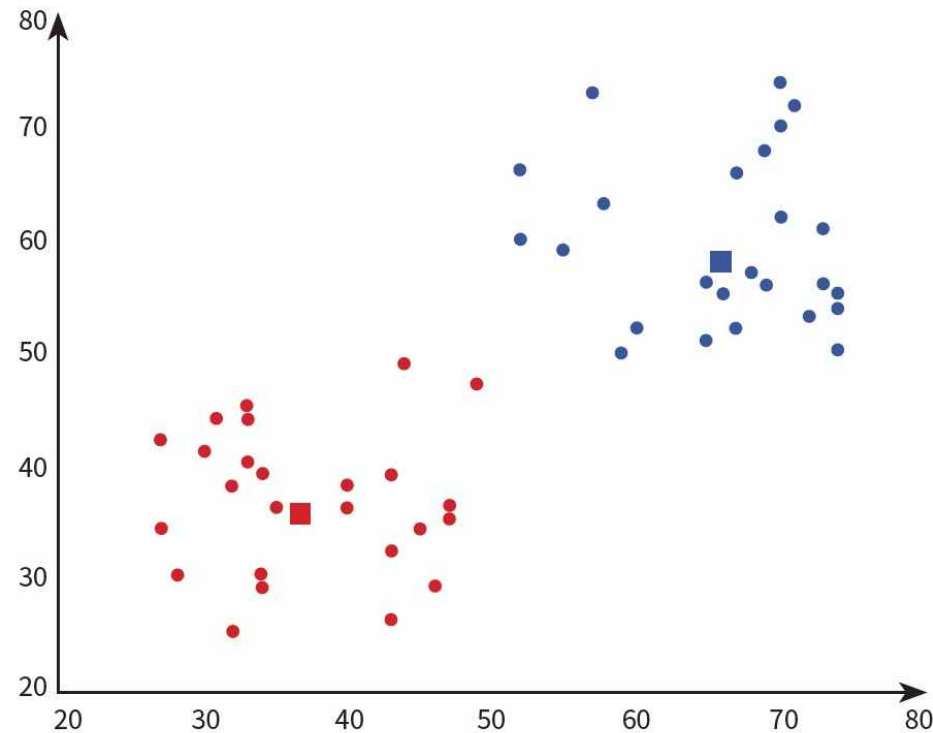
알고리즘 #3

- 모든 파란색 점과 빨간색 점의 평균을 따로 계산한다. 이 점이 클러스터의 새로운 중심이 된다.



알고리즘 #4

- 2개의 중심점의 위치가 변하지 않을 때까지 2단계와 3단계를 반복한다.



e.g.

- 다음과 같은 데이터가 있다고 하자. $k=2$ 이고 첫 번째 클러스터의 초기 중심은 **P2**, 두 번째 클러스터의 초기 중심은 **P8**로 한다. K-means 알고리즘의 한 단계만을 수행하라.

번호	X	Y
P1	2	3
P2	3	1
P3	4	2
P4	11	5
P5	12	4
P6	12	6
P7	7	5
P8	8	4
P9	8	6

05 sklearn을 이용한 K-means 클러스터링

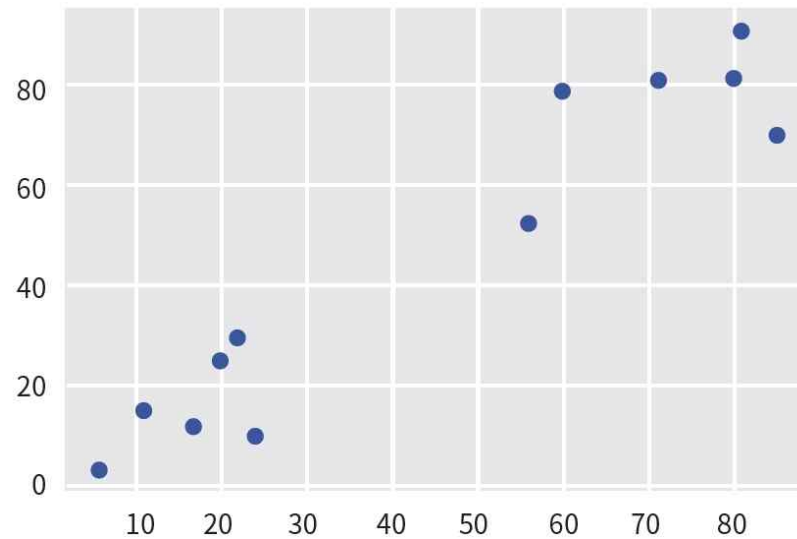
- 여기서는 **sklearn** 라이브러리를 이용하여 **K-means** 클러스터링 알고리즘을 실습해보자.

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.cluster import Kmeans

X = np.array([
    [6,3], [11,15], [17,12], [24,10], [20,25], [22,30],
    [85,70], [71,81], [60,79], [56,52], [81,91], [80,81]])

plt.scatter(X[:,0],X[:,1])
```

sklearn을 이용한 K-means 클러스터링



sklearn을 이용한 K-means 클러스터링

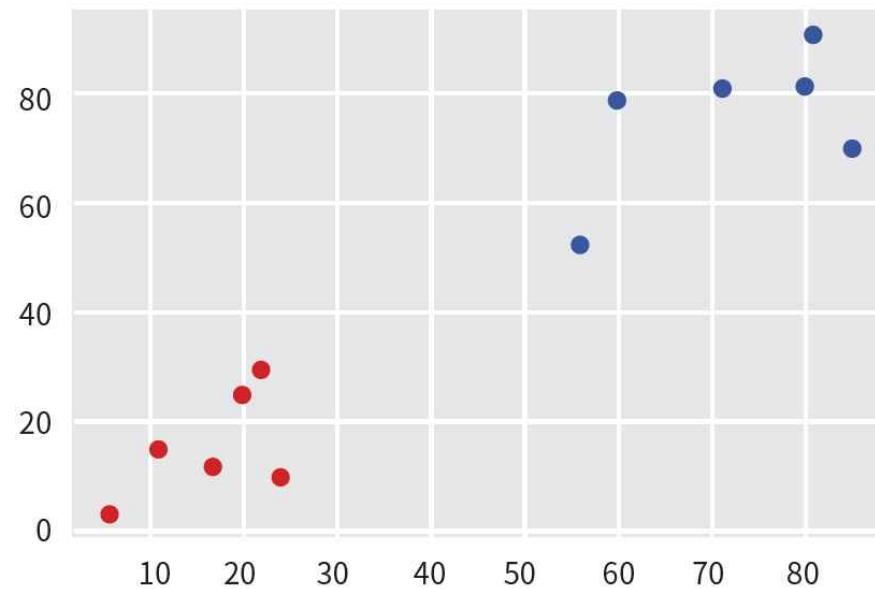
```
kmeans = KMeans(n_clusters=2)
kmeans.fit(X)

print(kmeans.cluster_centers_)
```

```
[[72.16666667 75.66666667]
 [16.66666667 15.83333333]]
```

sklearn을 이용한 K-means 클러스터링

```
print(kmeans.labels_)  
plt.scatter(X[:,0],X[:,1], c=kmeans.labels_, cmap='rainbow')
```

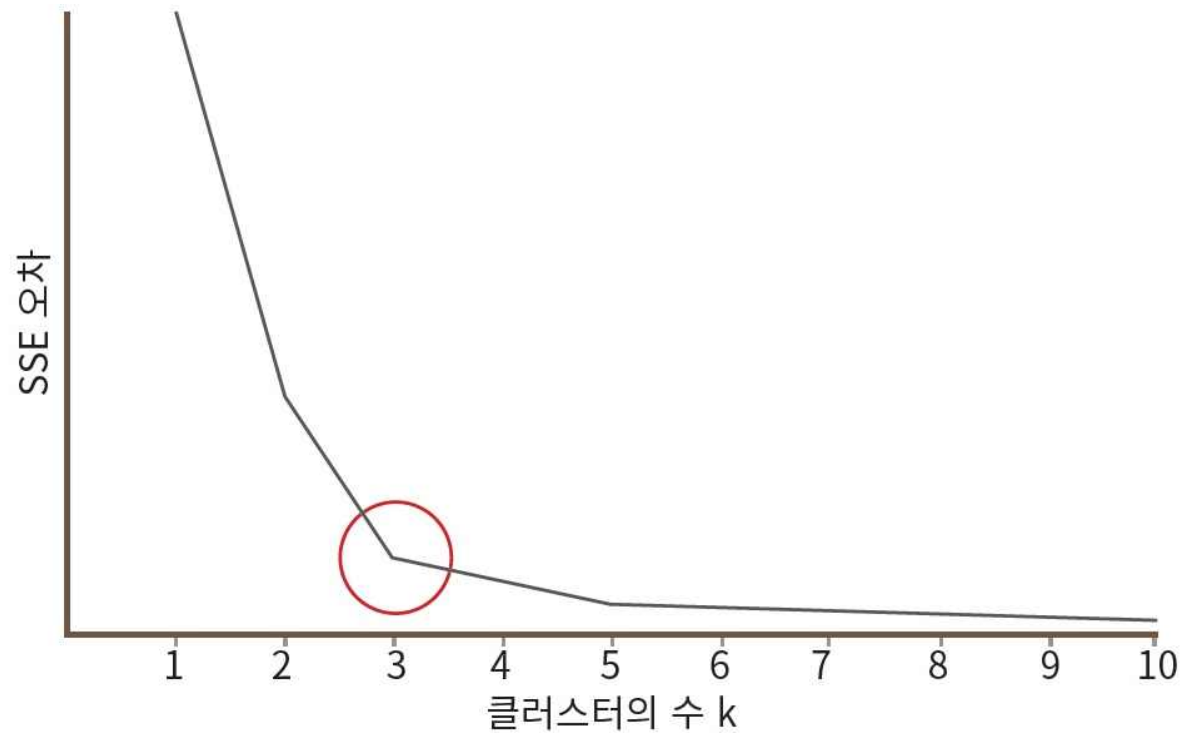


06 k를 결정하는 방법

- "팔꿈치" 방법(elbow method)에서는 k를 1부터 증가시키면서 K-means 클러스터링을 수행
 - 각 k의 값에 대하여 SSE(sum of squared errors)의 값을 계산

```
var sse = {};  
for (var k = 1; k <= maxK; ++k) {  
  sse[k] = 0;  
  clusters = kmeans(dataset, k);  
  clusters.forEach(function(cluster) {  
    mean = clusterMean(cluster);           // 각 cluster의 평균을 구하고  
    cluster.forEach(function(datapoint) {  
      sse[k] += Math.pow(datapoint - mean, 2); // cluster안의 에러 제곱합 계산  
    });  
  });  
}
```

k를 결정하는 방법



k를 결정하는 방법

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.cluster import KMeans

X = np.array([
    [6,3], [11,15], [17,12], [24,10], [20,25], [22,30],
    [85,70], [71,81], [60,79], [56,52], [81,91], [80,81]])

# 첫 번째 그래프: 데이터 포인트 시각화
plt.figure(figsize=(12, 5))

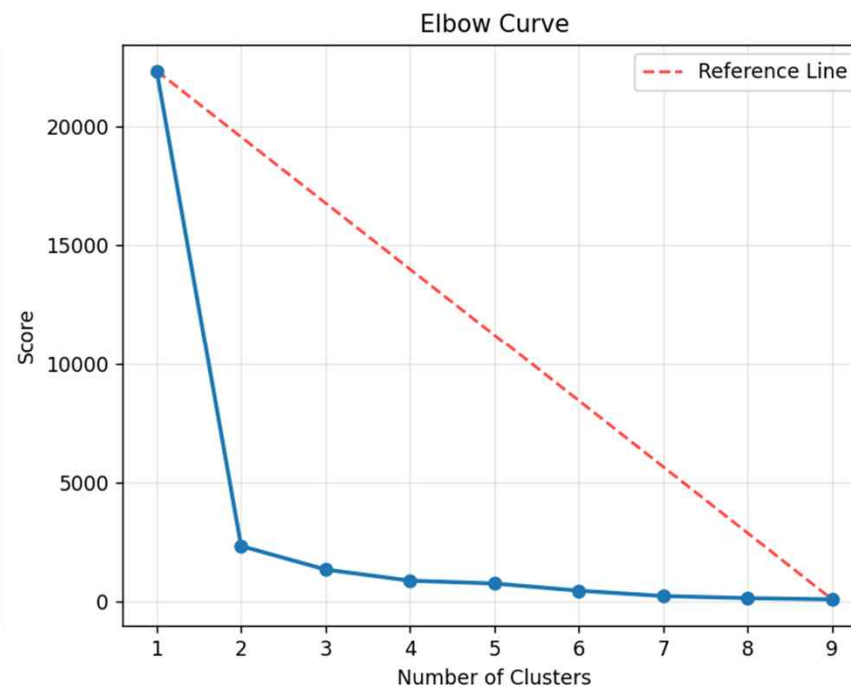
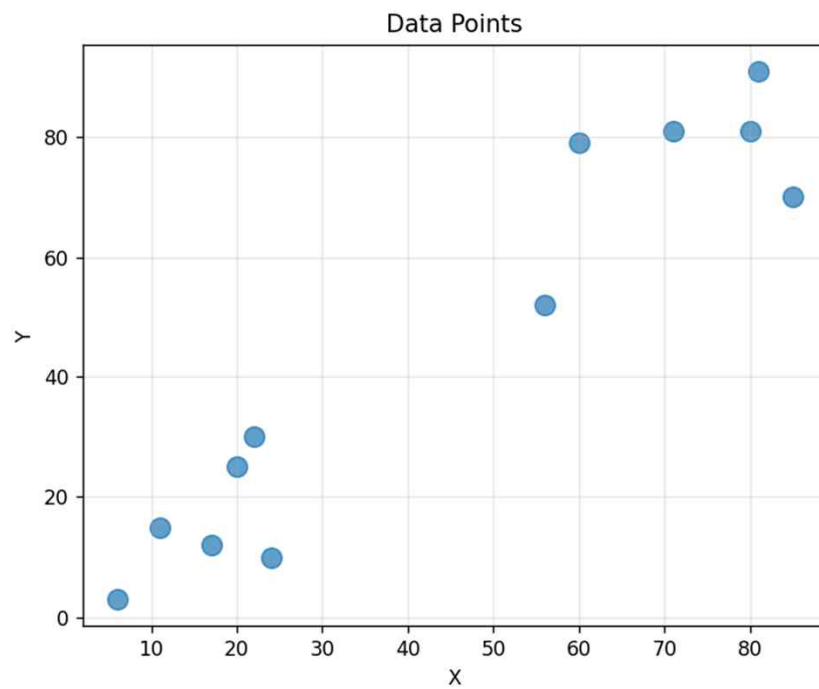
plt.subplot(1, 2, 1)
plt.scatter(X[:,0], X[:,1], s=100, alpha=0.7)
plt.xlabel('X')
plt.ylabel('Y')
plt.title('Data Points')
plt.grid(True, alpha=0.3)
```

```
# 두 번째 그래프: 엘보우 커브
plt.subplot(1, 2, 2)
n_clusters = range(1, 10)
kmeans = [KMeans(n_clusters=i) for i in n_clusters]

# 모든 샘플에 대하여 제곱 오차를 계산하여 리스트에 추가한다.
score = [kmeans[i].fit(X).inertia_ for i in range(len(kmeans))]
plt.plot([min(n_clusters), max(n_clusters)], [max(score), min(score)], color='red',
linestyle='--', alpha=0.7, label='Reference Line')
plt.plot(n_clusters, score, marker='o', linewidth=2, markersize=6)
plt.xlabel('Number of Clusters')
plt.ylabel('Score')
plt.title('Elbow Curve')
plt.grid(True, alpha=0.3)
plt.legend()

plt.tight_layout()
plt.show()
```

k를 결정하는 방법



Lab: K-means 알고리즘 실습

- sklearn.datasets의 make_blobs() 함수 이용

```
import matplotlib.pyplot as plt
import seaborn as sns; sns.set()
import numpy as np
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

X, y_true = make_blobs(n_samples=300, centers=4,
                        cluster_std=0.60, random_state=0)

# 그림 크기 설정 및 서브플롯 생성
plt.figure(figsize=(12, 5))

# 첫 번째 서브플롯: 원본 데이터
plt.subplot(1, 2, 1) # 1행 2열의 첫 번째 그래프
plt.scatter(X[:, 0], X[:, 1], s=50)
plt.title('Original Data')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.grid(True, alpha=0.3)
```


Lab: K-means 알고리즘 실습

```
# 두 번째 서브플롯: 클러스터링 결과
plt.subplot(1, 2, 2) # 1행 2열의 두 번째 그래프
# K-means 클러스터링 수행
kmeans = KMeans(n_clusters=4)
kmeans.fit(X)
y_kmeans = kmeans.predict(X)

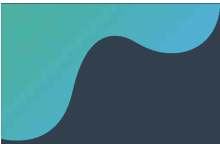
# 클러스터링 결과 시각화
plt.scatter(X[:, 0], X[:, 1], c=y_kmeans, s=50, cmap='viridis')

# 클러스터 중심점 표시
centers = kmeans.cluster_centers_
plt.scatter(centers[:, 0], centers[:, 1], c='black', s=200, alpha=0.5)
plt.title('K-means Clustering Result')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.grid(True, alpha=0.3)

# 그래프 간 간격 조정 및 출력
plt.tight_layout()
plt.show()
```

Lab: K-means 알고리즘 실습





Contents

CHAPTER 11 비지도 학습

SECTION 11-1 군집 알고리즘

SECTION 11-2 k-평균

SECTION 11-3 주성분 분석





CHAPTER 11 비지도 학습

비슷한 과일끼리 모으자!

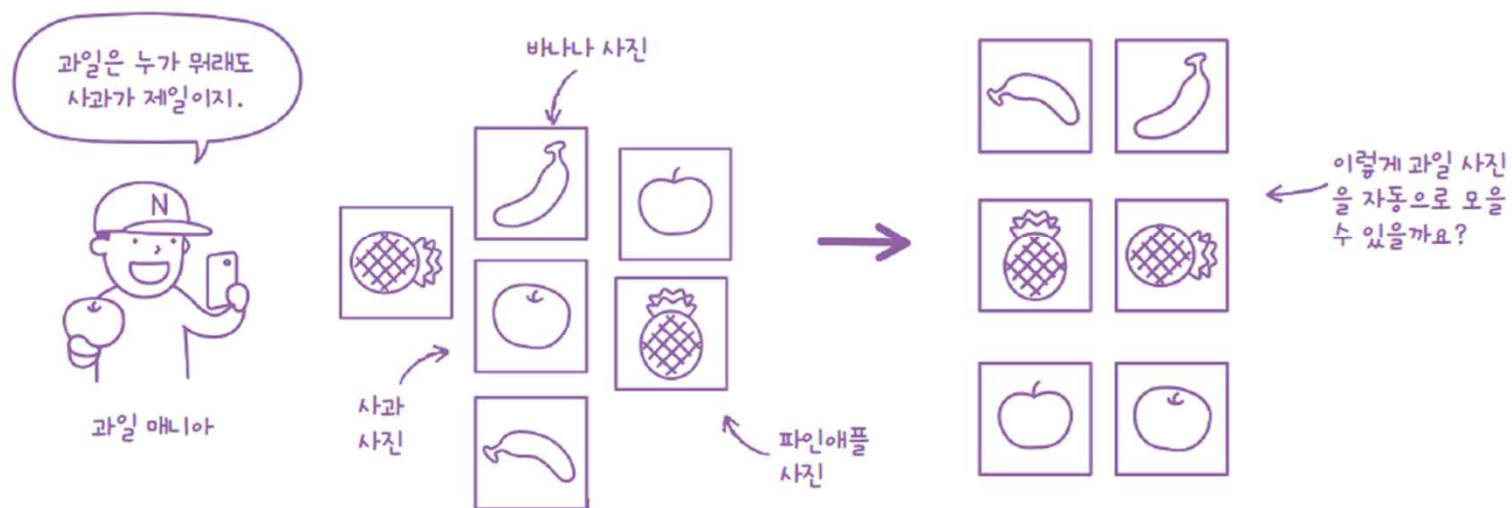
학습목표

- 타깃이 없는 데이터를 사용하는 비지도 학습과 대표적인 알고리즘을 소개합니다.
- 대표적인 군집 알고리즘인 k-평균과 DBSCAN을 배웁니다.
- 대표적인 차원 축소 알고리즘인 주성분 분석(PCA)을 배웁니다.

SECTION 11-1 군집 알고리즘(1)

◦ 타깃을 모르는 비지도 학습

- 타깃을 모르는 사진을 종류별로 분류
- 타깃이 없을 때 사용하는 머신러닝 알고리즘이 비지도 학습(unsupervised learning)



SECTION 11-1 군집 알고리즘(2)

◦ 과일 사진 데이터 준비하기

- ⑩ 준비한 과일 데이터는 사과, 바나나, 파인애플을 담고 있는 흑백 사진
- ⑩ 이 데이터는 넘파이 배열의 기본 저장 포맷인 npy 파일로 저장되어 있음
- ⑩ 코랩에서 다음 명령을 실행해 파일을 다운로드

```
!wget https://bit.ly/fruits_300_data -O fruits_300.npy
```

- ⑩ 넘파이와 맷플롯립 패키지를 임포트

```
import numpy as np  
import matplotlib.pyplot as plt
```

- ⑩ 넘파이에서 load() 메서드에 파일 이름을 전달하여 npy 파일을 로드

```
fruits = np.load('fruits_300.npy')
```

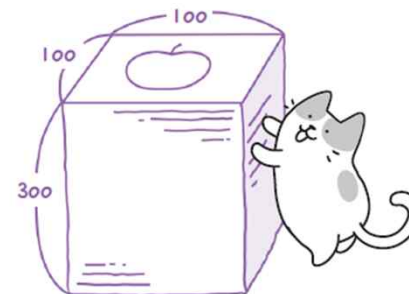
- ⑩ fruits 배열의 크기를 확인

```
print(fruits.shape)
```



(300, 100, 100)

(샘플갯수, 이미지 높이, 이미지 너비)



SECTION 11-1 군집 알고리즘(3)

과일 사진 데이터 준비하기

- ⑩ 첫 번째 이미지의 첫 번째 행에 들어 있는 픽셀 100개 값을 출력
흑백 사진을 담고 있으므로 0~255의 정숫값을 가짐

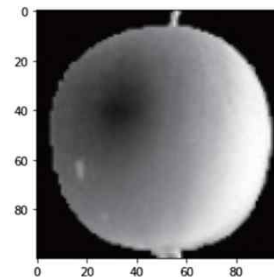
```
print(fruits[0, 0, :])
```

→

1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	2	1
2	2	2	2	2	2	1	1	1	1	1	1	1	1	2	3	2	1
2	1	1	1	1	2	1	3	2	1	3	1	4	1	2	5	5	5
19	148	192	117	28	1	1	2	1	4	1	1	3	1	1	1	1	1
2	2	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

- ⑩ 맷플롯립의 imshow() 함수를 사용하여 넘파이 배열로 저장된 이미지를 그림
흑백 이미지이므로 cmap 매개변수를 'gray'로 지정

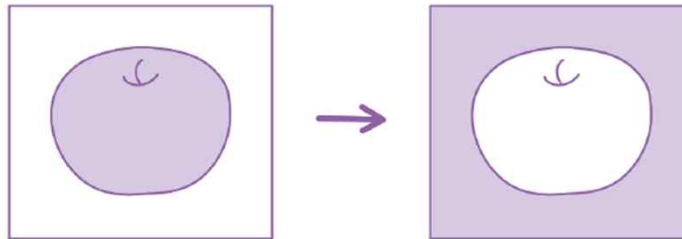
```
plt.imshow(fruits[0], cmap='gray')  
plt.show()
```



SECTION 11-1 군집 알고리즘(4)

◦ 과일 사진 데이터 준비하기

- ⑩ 흑백 이미지는 사진으로 찍은 이미지를 넘파이 배열로 변환할 때 반전시킨 것
- ⑩ 사진의 흰 바탕(높은 값)은 검은색(낮은 값)으로 만들고 실제 사과가 있어 짙은 부분(낮은 값)은 밝은색(높은 값)으로 변환
- ⑩ 흰색 바탕은 우리에게 중요하지 않지만 컴퓨터는 255에 가까운 바탕에 집중. 따라서 바탕을 검게 만들고 사진에 짙게 나온 사과를 밝은색으로 만들



✚ 여기서 잠깐 컴퓨터는 왜 255에 가까운 바탕에 집중하나?

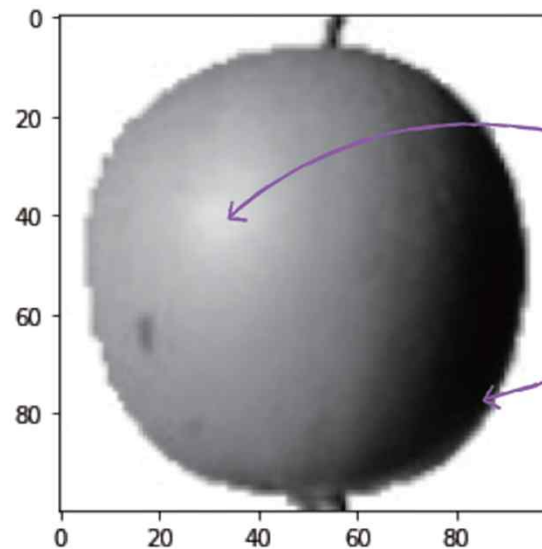
- 알고리즘이 어떤 출력을 만들기 위해 곱셈, 덧셈을 수행
- 픽셀값이 0이면 출력도 0이 되어 의미가 없음
- 픽셀값이 높으면 출력값도 커지기 때문에 의미를 부여하기 좋음

SECTION 11-1 군집 알고리즘(5)

- 과일 사진 데이터 준비하기

⑩ cmap 매개변수를 'gray_r'로 지정하면 다시 반전하여 우리 눈에 보기 좋게 출력

```
plt.imshow(fruits[0], cmap='gray_r')  
plt.show()
```



밝은 부분은 0에 가깝습니다.

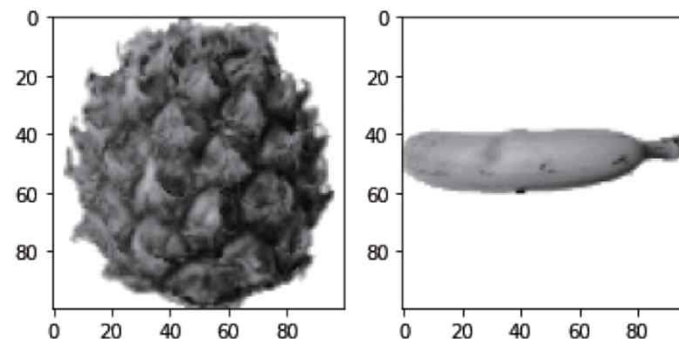
짙은 부분은 255에 가깝습니다.

SECTION 11-1 군집 알고리즘(6)

과일 사진 데이터 준비하기

⑩ 바나나와 파인애플 이미지도 출력

```
fig, axs = plt.subplots(1, 2)
axs[0].imshow(fruits[100], cmap='gray_r')
axs[1].imshow(fruits[200], cmap='gray_r')
plt.show()
```



⑩ 맷플롯립의 `subplots()` 함수를 사용하면 여러 개의 그래프를 배열처럼 쌓을 수 있도록 도와줌

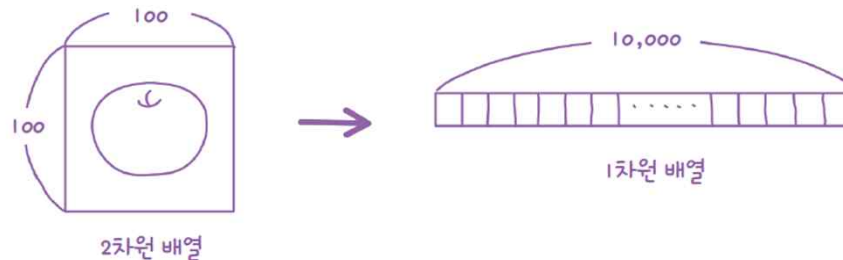
⑩ `subplots()` 함수의 두 매개변수는 그래프를 쌓을 행과 열을 지정

⑩ 위에서는 `subplots(1, 2)`처럼 하나의 행과 2개의 열을 지정

SECTION 11-1 군집 알고리즘(7)

픽셀값 분석하기

- ⑩ 사용하기 쉽게 fruits 데이터를 사과, 파인애플, 바나나로 각각 나누기
- ⑩ 넘파이 배열을 나눌 때 100×100 이미지를 펼쳐서 길이가 10,000인 1차원 배열로 만들기
 - 이렇게 펼치면 이미지로 출력하긴 어렵지만 배열을 계산할 때 편리



- ⑩ fruits 배열에서 순서대로 100개씩 선택하기 위해 슬라이싱 연산자를 사용. 그다음 reshape() 메서드를 사용해 두 번째 차원(100)과 세 번째 차원(100)을 10,000으로 합침

```
apple = fruits[0:100].reshape(-1, 100*100)
pineapple = fruits[100:200].reshape(-1, 100*100)
banana = fruits[200:300].reshape(-1, 100*100)
```

- ⑩ 이제 apple, pineapple, banana 배열의 크기는 (100, 10000)이 됨

SECTION 11-1 군집 알고리즘(8)

- 픽셀값 분석하기

⑩ apple 배열의 mean() 메서드로 각 샘플의 픽셀 평균값을 계산

```
print(apple.mean(axis=1))
```



```
[ 88.3346  97.9249  87.3709  98.3703  92.8705  82.6439  94.4244  95.5999
 90.681   81.6226  87.0578  95.0745  93.8416  87.017   97.5078  87.2019
 88.9827 100.9158  92.7823 100.9184 104.9854  88.674   99.5643  97.2495
 94.1179  92.1935  95.1671  93.3322 102.8967  94.6695  90.5285  89.0744
 97.7641  97.2938 100.7564  90.5236 100.2542  85.8452  96.4615  97.1492
 90.711   102.3193  87.1629  89.8751  86.7327  86.3991  95.2865  89.1709
 96.8163  91.6604  96.1065  99.6829  94.9718  87.4812  89.2596  89.5268
 93.799   97.3983  87.151   97.825  103.22   94.4239  83.6657  83.5159
102.8453  87.0379  91.2742 100.4848  93.8388  90.8568  97.4616  97.5022
 82.446   87.1789  96.9206  90.3135  90.565   97.6538  98.0919  93.6252
 87.3867  84.7073  89.1135  86.7646  88.7301  86.643   96.7323  97.2604
 81.9424  87.1687  97.2066  83.4712  95.9781  91.8096  98.4086 100.7823
101.556  100.7027  91.6098  88.8976]
```

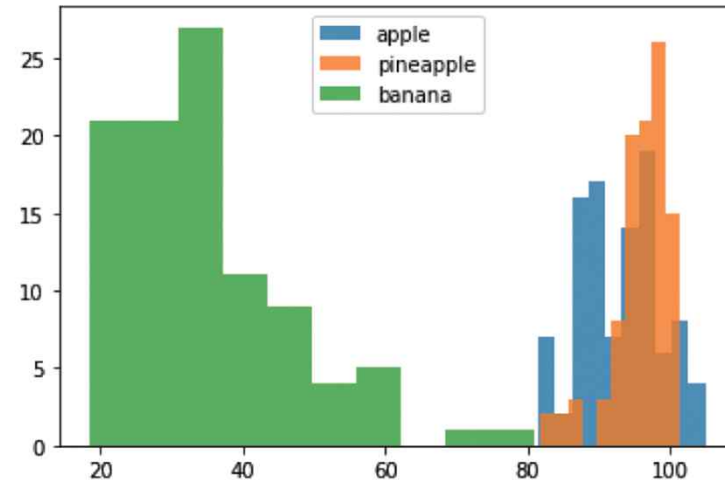
SECTION 11-1 군집 알고리즘(9)

픽셀값 분석하기

⑩ 맷플롯립의 hist() 함수로 평균값 분포 히스토그램을 구현

- legend() 함수를 사용해 어떤 과일의 히스토그램인지 범례 추가

```
plt.hist(np.mean(apple, axis=1), alpha=0.8)
plt.hist(np.mean(pineapple, axis=1), alpha=0.8)
plt.hist(np.mean(banana, axis=1), alpha=0.8)
plt.legend(['apple', 'pineapple', 'banana'])
plt.show()
```



⑩ 바나나는 픽셀 평균값만으로 사과나 파인애플과 확실히 구분됨

- 바나나는 사진에서 차지하는 영역이 작기 때문에 평균값이 작음

⑩ 반면 사과와 파인애플은 많이 겹쳐있어서 픽셀값만으로는 구분하기 쉽지 않음

- 사과나 파인애플은 대체로 형태가 동그랗고 사진에서 차지하는 크기도 비슷하기 때문

SECTION 11-1 군집 알고리즘(10)

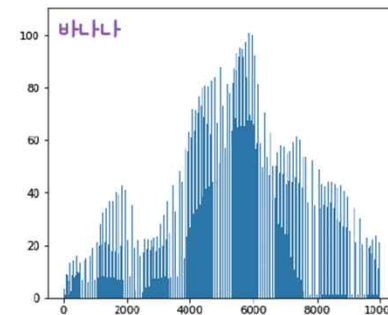
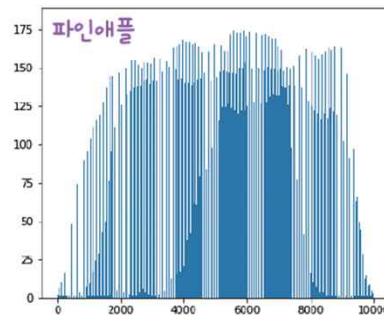
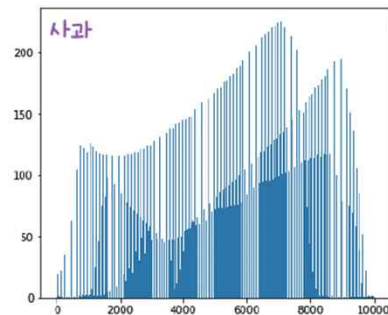
픽셀값 분석하기

⑩ 샘플의 평균값이 아니라 픽셀별 평균값 비교

- 픽셀의 평균을 계산하기 위해 axis=0으로 지정

⑩ 맷플롯립의 bar() 함수를 사용해 픽셀 10,000개에 대한 평균값을 막대그래프로 구현

```
fig, axs = plt.subplots(1, 3, figsize=(20,5))  
axs[0].bar(range(10000), np.mean(apple, axis=0))  
axs[1].bar(range(10000), np.mean(pineapple, axis=0))  
axs[2].bar(range(10000), np.mean(banana, axis=0))  
plt.show()
```

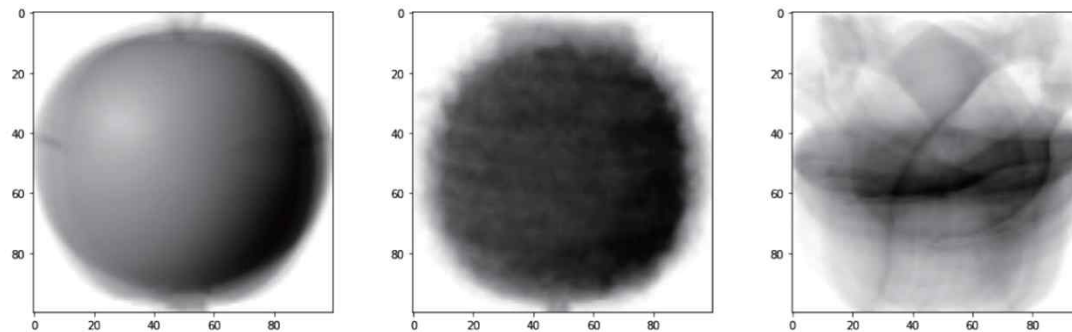


SECTION 11-1 군집 알고리즘(11)

- 픽셀값 분석하기

⑩ 픽셀 평균값을 100×100 크기로 바꿔서 이미지처럼 출력하여 앞의 그래프와 비교

```
apple_mean = np.mean(apple, axis=0).reshape(100, 100)
pineapple_mean = np.mean(pineapple, axis=0).reshape(100, 100)
banana_mean = np.mean(banana, axis=0).reshape(100, 100)
fig, axs = plt.subplots(1, 3, figsize=(20,5))
axs[0].imshow(apple_mean, cmap='gray_r')
axs[1].imshow(pineapple_mean, cmap='gray_r')
axs[2].imshow(banana_mean, cmap='gray_r')
plt.show()
```



SECTION 11-1 군집 알고리즘(12)

◦ 평균값과 가까운 사진 고르기

- ⑩ 사과 사진의 평균값인 apple_mean과 가장 가까운 사진 고르기
-3장에서 학습한 절댓값 오차를 사용

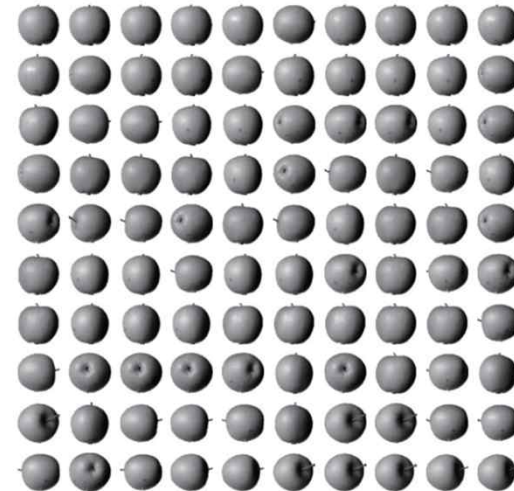
- ⑩ fruits 배열에 있는 모든 샘플에서 apple_mean을 뺀 절댓값의 평균을 계산

```
abs_diff = np.abs(fruits - apple_mean)
abs_mean = np.mean(abs_diff, axis=(1,2))
print(abs_mean.shape)
```

→ (300,)

- ⑩ apple_mean과 오차가 가장 작은 샘플 100개 고르기

```
apple_index = np.argsort(abs_mean)[:100]
fig, axs = plt.subplots(10, 10, figsize=(10,10))
for i in range(10):
    for j in range(10):
        axs[i, j].imshow(fruits[apple_index[i*10 + j]], cmap='gray_r')
        axs[i, j].axis('off')
plt.show()
```



- 군집(clustering): 비슷한 샘플끼리 그룹으로 모으는 작업
- 클러스터(cluster): 군집 알고리즘에서 만든 그룹

SECTION 11-1 군집 알고리즘(13)

◦ 비슷한 샘플끼리 모으기(문제해결 과정)

▪ 문제

⑩ 고객들이 올린 과일 사진을 자동으로 모으기

⑩ 어떤 과일 사진을 올릴지 미리 예상할 수 없기 때문에 타깃값을 준비하여 분류 모델을 훈련하기 어려움

▪ 해결

⑩ 비지도 학습: 타깃값이 없을 때 데이터에 있는 패턴을 찾거나 데이터 구조를 파악하는 머신러닝 방식

⑩ 타깃이 없기 때문에 알고리즘을 직접적으로 가르칠 수가 없고, 대신 알고리즘은 스스로 데이터가 어떻게 구성되어 있는지 분석

⑩ 대표적인 비지도 학습 문제는 '군집'

⑩ 군집은 비슷한 샘플끼리 그룹으로 모으는 작업

⑩ 이 절에서는 사진의 픽셀을 사용해 군집과 비슷한 작업을 수행, 샘플이 어떤 과일인지 미리 알고 있었기 때문에 사과 사진의 평균값을 알 수 있었음

⑩ 실제 비지도 학습에서는 타깃이 없는 사진을 사용

SECTION 11-1 마무리

- 키워드로 끝나는 핵심 포인트

- 비지도 학습은 머신러닝의 한 종류로 훈련 데이터에 타깃이 없음
 - ⑩ 타깃이 없기 때문에 외부의 도움 없이 스스로 유용한 무언가를 학습해야 함
 - ⑩ 대표적인 비지도 학습 작업은 군집, 차원 축소 등
- 히스토그램은 구간별로 값이 발생한 빈도를 그래프로 표시한 것
 - ⑩ 보통 x축이 값의 구간(계급)이고, y축은 발생 빈도(도수)
- 군집은 비슷한 샘플끼리 하나의 그룹으로 모으는 대표적인 비지도 학습 작업
 - ⑩ 클러스터: 군집 알고리즘으로 모은 샘플 그룹

SECTION 11-1 확인 문제

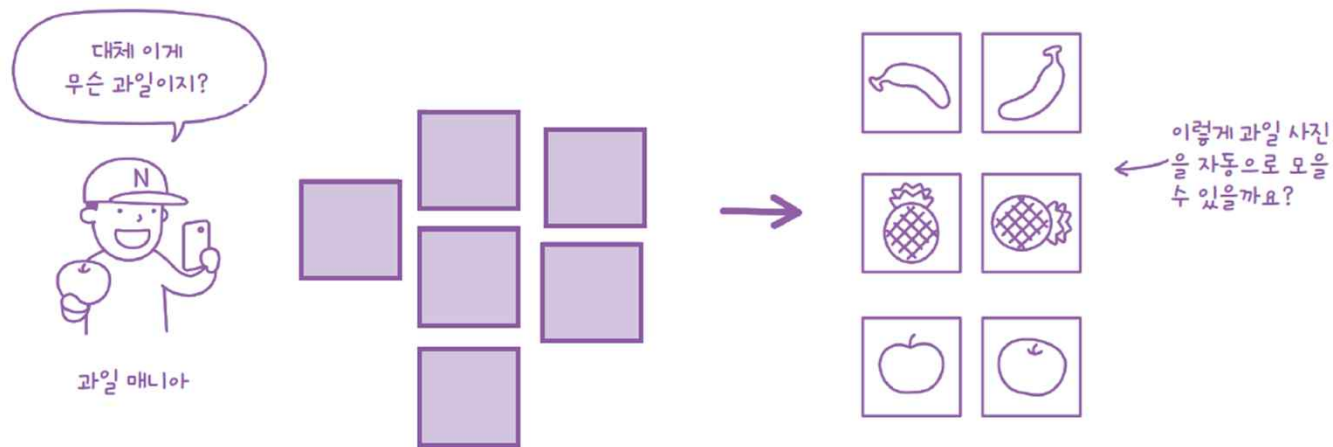
1. 히스토그램을 그릴 수 있는 맷플롯립 함수는?

- ① hist()
- ② scatter()
- ③ plot()
- ④ bar()

2. apple에서 했던 것처럼 바나나 사진의 평균 banana_mean과 비슷한 사진 100장을 찾아 출력하시오. 바나나 사진을 모두 찾을 수 있나?

SECTION 11-2 k-평균(1)

- k-평균(k-means) 군집 알고리즘이 평균값을 자동으로 찾아줌
- 이 평균값이 클러스터의 중심에 위치하기 때문에 클러스터 중심(cluster center) 또는 센트로이드(centroid)라고 함

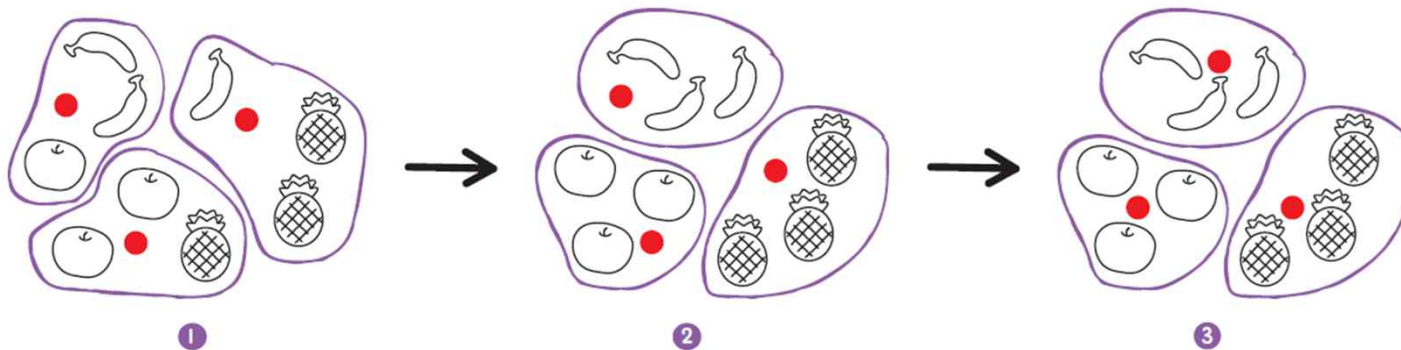


SECTION 11-2 k-평균(2)

◦ k-평균 알고리즘 소개

▪ k-평균 알고리즘의 작동 방식

1. 무작위로 k개의 클러스터 중심을 정함
2. 각 샘플에서 가장 가까운 클러스터 중심을 찾아 해당 클러스터의 샘플로 지정
3. 클러스터에 속한 샘플의 평균값으로 클러스터 중심을 변경
4. 클러스터 중심에 변화가 없을 때까지 2번으로 돌아가 반복



SECTION 11-2 k-평균(3)

◦ KMeans 클래스

⑩ 1절에서 사용했던 데이터셋을 사용

⑩ wget 명령으로 데이터를 다운로드

```
!wget https://bit.ly/fruits_300 -O fruits_300.npy
```

⑩ np.load() 함수를 사용해 npy 파일을 읽어 넘파이 배열을 준비

- k-평균 모델을 훈련하기 위해 (샘플 개수, 너비, 높이) 크기의 3차원 배열을 (샘플 개수, 너비×높이) 크기를 가진 2차원 배열로 변경

```
import numpy as np
fruits = np.load('fruits_300.npy')
fruits_2d = fruits.reshape(-1, 100*100)
```

⑩ k-평균 알고리즘으로 3개 클러스터로 군집

```
from sklearn.cluster import KMeans
km = KMeans(n_clusters=3, random_state=42)
km.fit(fruits_2d)
```

```
import urllib.request
import os
url = 'https://bit.ly/fruits_300'
filename = 'fruits_300.npy'
urllib.request.urlretrieve(url, filename)
```



SECTION 11-2 k-평균(4)

- KMeans 클래스

10 k-평균의 레이블 출력

```
print(km.labels_)
```



```
[0 0 0 0 0 2 0 0 0 0 0 0 0 0 0 0 0 2 0 0 0 0 0 0 0 0 0 0 0 0 0 0  
0 0 0 0 0 2 0 2 0 0 0 0 0 0 0 2 0 0 0 0 0 0 0 0 2 2 0 0 0 0 0 0 0 2 0  
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 2 0 0 0 0 0 0 0 0 2 2 2 2 2 2 2 2 2  
2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2  
2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2  
2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2  
1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1  
1 1 1 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1  
1 1 1 1]
```

⑩레이블 0, 1, 2로 모은 샘플의 개수 확인

```
print(np.unique(km.labels_, return_counts=True))
```



```
(array([0, 1, 2], dtype=int32), array([ 91, 98, 111]))
```



SECTION 11-2 k-평균(5)

- KMeans 클래스

⑩ 각 클러스터가 어떤 이미지를 나타냈는지 그림으로 출력하기 위해 간단한 유틸리티 함수 draw_fruits() 사용

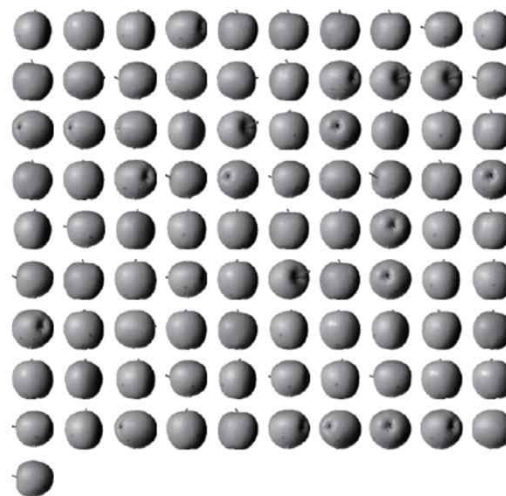
```
import matplotlib.pyplot as plt
def draw_fruits(arr, ratio=1):
    n = len(arr) # n은 샘플 개수입니다
    # 한 줄에 10개씩 이미지를 그립니다. 샘플 개수를 10으로 나누어 전체 행 개수를 계산합니다
    rows = int(np.ceil(n/10))
    # 행이 1개이면 열의 개수는 샘플 개수입니다. 그렇지 않으면 10개입니다
    cols = n if rows < 2 else 10
    fig, axs = plt.subplots(rows, cols,
                             figsize=(cols*ratio, rows*ratio), squeeze=False)
    for i in range(rows):
        for j in range(cols):
            if i*10 + j < n: # n 개까지만 그립니다
                axs[i, j].imshow(arr[i*10 + j], cmap='gray_r')
                axs[i, j].axis('off')
    plt.show()
```


SECTION 11-2 k-평균(6)

◦ KMeans 클래스

- ⑩ 앞의 함수를 사용해 레이블이 0인 과일 사진을 모두 그리기
 - 불리언 인덱싱 적용하면 True인 위치의 원소만 모두 추출

```
draw_fruits(fruits[km.labels_==0])
```



▲ 레이블 0으로 클러스터링된 91개의 이미지를 모두 출력

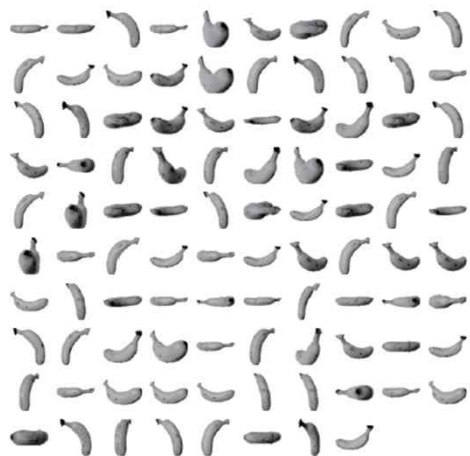
SECTION 11-2 k-평균(7)

◦ KMeans 클래스

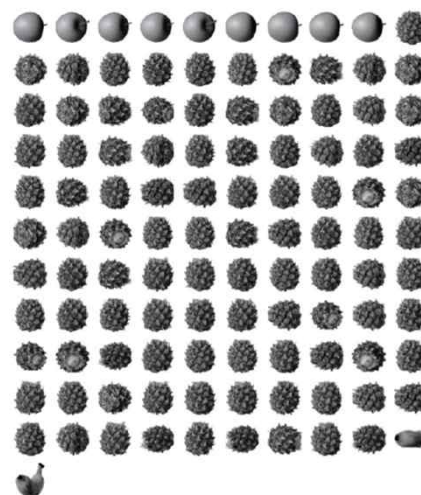
⑩ 앞의 함수를 사용해 레이블이 0인 과일 사진을 모두 그리기

- 다른 두 클러스터의 이미지 출력

```
draw_fruits(fruits[km.labels_==1])
```



```
draw_fruits(fruits[km.labels_==2])
```

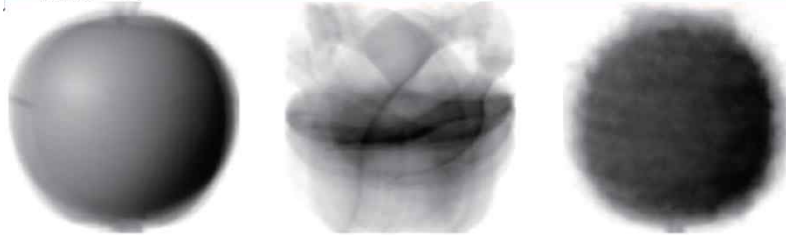


SECTION 11-2 k-평균(8)

◦ 클러스터 중심

- ⑩ KMeans 클래스가 최종적으로 찾은 클러스터 중심(10000차원)은 `cluster_centers_` 속성에 저장됨. 이 배열은 `fruits_2d` 샘플의 클러스터 중심이기 때문에 이미지로 출력하려면 100×100 크기의 2차원 배열로 바꿔야 함

```
draw_fruits(km.cluster_centers_.reshape(-1, 100, 100),  
            ratio=3)
```



- ⑩ 훈련 데이터 샘플에서 클러스터 중심까지 거리로 변환해 주는 `transform()` 메서드
- ⑩ 인덱스가 100인 샘플에 `transform()` 메서드를 적용
 - 슬라이싱 연산자를 사용해서 (1, 10000) 크기의 배열 하나를 전달

```
print(km.transform(fruits_2d[100:101])) → [[5267.70439881 8837.37750892 3393.8136117 ]]
```

SECTION 11-2 k-평균(9)

◦ 클러스터 중심

- ⑩ 가장 가까운 클러스터 중심을 예측 클래스로 출력하는 predict() 메서드

```
print(km.predict(fruits_2d[100:101]))
```

 → [2]

- ⑩ 클러스터 중심을 그려보았을 때 레이블 2는 파인애플. 이미지로 확인

```
draw_fruits(fruits[100:101])
```

 →

- ⑩ 알고리즘이 반복한 횟수는 KMeans 클래스의 n_iter_ 속성에 저장됨

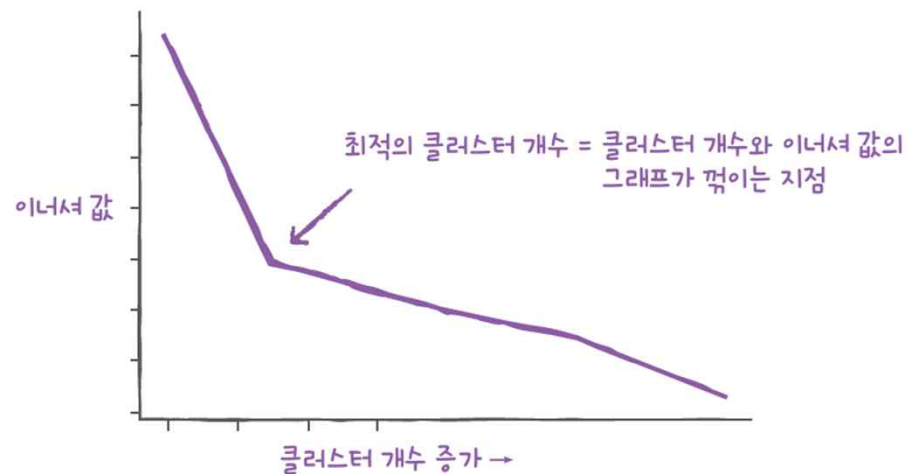
```
print(km.n_iter_)
```

 → 3

SECTION 11-2 k-평균(10)

◦ 최적의 k 찾기

- k-평균 알고리즘의 단점 중 하나는 클러스터 개수를 사전에 지정해야 한다는 점
- 엘보우(elbow) 방법: 적절한 클러스터 개수를 찾기 위한 대표적인 방법
 - ⑩ 이너셔(inertia): k-평균 알고리즘은 클러스터 중심과 클러스터에 속한 샘플 사이 거리의 제곱 합
 - ⑩ 이너셔는 클러스터에 속한 샘플이 얼마나 가깝게 모여 있는지를 나타내는 값



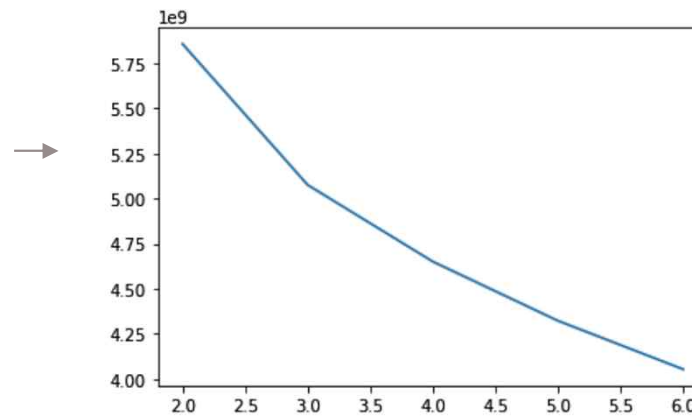
SECTION 11-2 k-평균(11)

◦ 최적의 k 찾기

- 엘보우(elbow) 방법: 적절한 클러스터 개수를 찾기 위한 대표적인 방법

⑩ 과일 데이터셋을 사용해 이너셔 계산하기

```
inertia = []  
for k in range(2, 7):  
    km = KMeans(n_clusters=k, random_  
state=42)  
    km.fit(fruits_2d)  
    inertia.append(km.inertia_)  
plt.plot(range(2, 7), inertia)  
plt.show()
```



- ▲ 이 그래프에서는 꺾이는 지점이 뚜렷하지는 않지만, $k=3$ 에서 그래프의 기울기가 조금 바뀐 것을 볼 수 있음
엘보우 지점보다 클러스터 개수가 많아지면 이너셔의 변화가 줄어들면서 군집 효과도 줄어들지만 이 그래프에서는 이런 지점이 명확하지는 않음

SECTION 11-2 k-평균(12)

- 과일을 자동으로 분류하기(문제해결 과정)

- 문제

- ⑩ 타깃값을 모르는 척하고 자동으로 사진을 클러스터로 모을 수 있는 군집 알고리즘이 필요

- 해결

- ⑩ k-평균 알고리즘

- ⑩ transform() 메서드: 각 샘플에서 각 클러스터까지의 거리를 하나의 특성으로 활용

- ⑩ predict() 메서드에서 새로운 샘플에 대해 가장 가까운 클러스터를 예측값으로 출력

- ⑩ 최적의 클러스터 개수 k를 알아내기 위해 클러스터가 얼마나 밀집되어 있는지 나타내는 이너셔를 사용

- ⑩ 이너셔가 더 이상 크게 줄어들지 않는다면 클러스터 개수를 더 늘리는 것은 효과가 없음

- ⑩ 엘보우 방법: 클러스터 개수를 늘리면서 반복하여 KMeans 알고리즘을 훈련하고 이너셔가 줄어드는 속도가 꺾이는 지점을 최적의 클러스터 개수로 결정

SECTION 11-2 마무리(1)

- 키워드로 끝내는 핵심 포인트

- k-평균 알고리즘은 처음에 랜덤하게 클러스터 중심을 정하고 클러스터를 만듦
 - ⑩ 클러스터의 중심을 이동하고 다시 클러스터를 만드는 식으로 반복해서 최적의 클러스터를 구성하는 알고리즘
- 클러스터 중심은 k-평균 알고리즘이 만든 클러스터에 속한 샘플의 특성 평균값
 - ⑩ 센트로이드(centroid)라고도 부름
 - ⑩ 가장 가까운 클러스터 중심을 샘플의 또 다른 특성으로 사용하거나 새로운 샘플에 대한 예측으로 활용
- 엘보우 방법은 최적의 클러스터 개수를 정하는 방법 중 하나
 - ⑩ 이너셔는 클러스터 중심과 샘플 사이 거리의 제곱 합
 - ⑩ 클러스터 개수에 따라 이너셔 감소가 꺾이는 지점이 적절한 클러스터 개수 k 가 될 수 있음
 - ⑩ 이 그래프의 모양을 따서 엘보우 방법이라고 부름

SECTION 11-2 마무리(2)

- 핵심 패키지와 함수
 - scikit-learn
 - ⑩ Kmeans: k-평균 알고리즘 클래스

SECTION 11-2 확인 문제

1. k-평균 알고리즘에서 클러스터를 표현하는 방법이 아닌것은?

- ① 클러스터에 속한 샘플의 평균
- ② 클러스터 중심
- ③ 센트로이드
- ④ 클러스터에 속한 샘플 ~~개수~~

2. k-평균에서 최적의 클러스터 개수는 어떻게 정할 수 있나?

- ① 엘보우 방법을 사용해 이너셔의 감소 정도가 꺾이는 클러스터 개수를 찾는다 팔꿈치
- ② 랜덤하게 클러스터 개수를 정해서 k-평균 알고리즘을 훈련하고 가장 낮은 이너셔가 나오는 클러스터 개수를 찾는다
- ③ 훈련 데이터를 모두 조사하여 몇 개의 클러스터가 나올 수 있는지 직접 확인
- ④ 교차 검증을 사용하여 최적의 클러스터 개수를 찾는다

Summary

- k-Nearest Neighbor(kNN) 알고리즘은 학습 시에 교사가 존재하는 “지도 학습”에 속한다. *regressor*
- kNN 알고리즘은 새로운 데이터를 가장 가까운 이웃 클래스로 할당한다. k는 홀수로 하는 것이 좋다.
- kNN 알고리즘은 어떤 종류의 학습이나 준비 시간이 필요 없지만 가장 가까운 이웃을 찾기 위해, 많은 메모리 공간과 계산 시간이 필요하다.
- K-means 알고리즘은 비지도 학습이다.
- K-means 알고리즘은 각 클러스터의 중심 (centroid)과 클러스터 내의 데이터와의 거리의 제곱합을 비용 함수로 정하고, 이 함수값을 최소화하는 방향으로 각 데이터의 소속 클러스터를 업데이트 해 줌으로써 클러스터링을 수행하게 된다.

Q & A

